Дисциплина «Проектирование архитектуры ПО»

Кафедра корпоративных информационных систем

ОТЧЁТ

по лабораторной работе №1

Тема: Аналитический запрос к распределённой структуре хранения через API Gateway с JWT-аутентификацией

Выполнил: студент группы БСБО-01-23

Судовский Андрей Леонидович

Москва, 2026

СОДЕРЖАНИЕ

1. Цель работы 4

2. Задание 4

3. Архитектура решения 5

3.1. Модель C4 5

3.2. Диаграммы потоков данных (нотация Йордана–Коуда) 6

3.3. API шлюза и аутентификация 8

4. Структура хранения, задействованная в выборке 8

5. Обоснование способа выборки 9

5.1. Шаг 1 — Elasticsearch: лекции с термином 9

5.2. Шаг 2 — PostgreSQL: посещаемость за период 10

5.3. Шаг 3 — Redis: карточки студентов 11

5.4. Порядок шагов 12

6. Реализация и развёртывание 12

7. Протокол тестирования 13

7.1. Развёртывание 14

7.2. Аутентификация и защита методов 14

7.3. Последовательность исполнения запроса 16

7.4. Проверка каждого шага отдельно 18

7.5. Независимая сверка результата 20

7.6. Граничные случаи 21

7.7. Журналы контейнеров 22

7.8. Вызов через Swagger UI 23

8. Выводы 25

Список использованных источников 26

Приложение А. Сервис ЛР №1: app\lab1\query.py 28

Приложение Б. Сервис ЛР №1: app\lab1\main.py 30

Приложение В. API Gateway: app\gateway\main.py 31

Приложение Г. JWT и пароли: app\common\security.py, app\common\web.py 32

Приложение Д. Dockerfile, docker-compose.yml, requirements.txt 33

Приложение Е. Независимая сверка: tests\verify.py 34

1. Цель работы

Реализовать аналитический запрос к структуре хранения практической работы №3 в виде отдельного сервиса в контейнере, опубликовать его через API Gateway с аутентификацией по токену JWT, обосновать выбор хранилищ и порядок выборки и подтвердить правильность результата протоколом тестирования.

2. Задание

«Выполнить запрос к структуре хранения информации о группах учащихся, курсах обучения, лекционной программе и составу лекционных курсов и практических занятий, а также структуре связей между курсами, специальностями, студентами кафедры и данными о посещении студентами занятий, для извлечения отчета о 10 студентах с минимальным процентом посещения лекций, содержащих заданный термин или фразу, за определенный период обучения. Состав полей должен включать полную информацию о студенте, процент посещения, период отчета, термин в занятиях курса».

Общие требования к лабораторным работам: источник данных — структура практической работы №3; архитектура изображается в нотациях C4 и DFD (Йордана–Коуда); система содержит API Gateway с методами вызова лабораторных работ по HTTP с входными параметрами и аутентификацией по токену JWT; каждая лабораторная работа — отдельный сервис в контейнере с HTTP-методом запуска. Отчёт содержит описание структуры хранения, задействованных хранилищ, обоснование способа выборки и протокол тестирования в виде последовательности исполнения с результатами каждого шага.

Работа выполнена по заданию [1] на структуре данных практической работы №3 [2].

Таблица 1 — Входные параметры метода GET /api/lab1/report

Параметр запроса	**Назначение**	**Пример**

`term`	термин или фраза (2–200 символов), ищется во всех словоформах	масштабирование
`date_from`, `date_to`	период обучения, даты занятий включительно	2026-02-09 … 2026-04-19
`limit`	размер отчёта, по умолчанию 10 (1–100)	10
`trace`	вернуть шаги исполнения с результатами и временем	true

3. Архитектура решения

3.1. Модель C4

Архитектура описана в модели C4 [3]. На уровне контекста (рисунок 1) пользователь API — преподаватель или сотрудник кафедры — обращается к аналитической системе кафедры, а она читает данные из хранилищ стенда практической работы №1, заполненных по структуре практической работы №3. Хранилища — внешняя система: их же используют остальные лабораторные работы, а изменения в них поступают из основного хранилища.

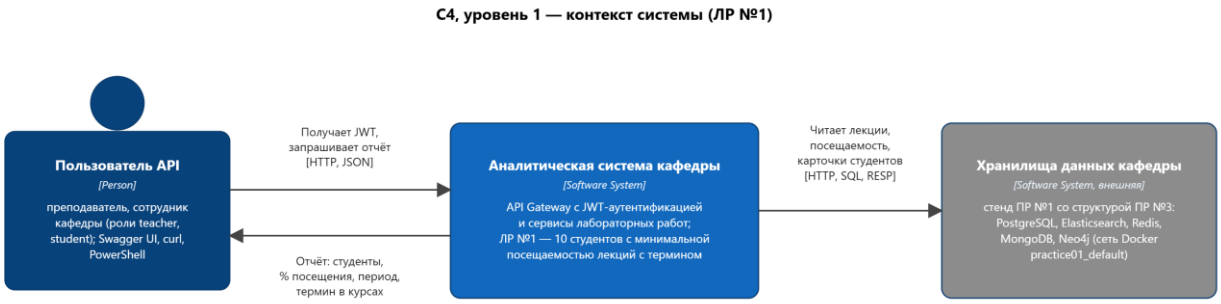


Рисунок 1 — C4, уровень 1: контекст системы

На уровне контейнеров (рисунок 2) система состоит из двух контейнеров Docker, собранных из одного образа `papo-lab:1.0` (Python 3.12, FastAPI [10], uvicorn): API Gateway [5] — единственная точка входа с опубликованным портом 8000, и сервис ЛР №1, доступный только внутри сети Docker `practice01_default`. В выборке участвуют три хранилища: Elasticsearch, PostgreSQL и Redis; MongoDB и Neo4j для этого запроса не нужны (раздел 5).

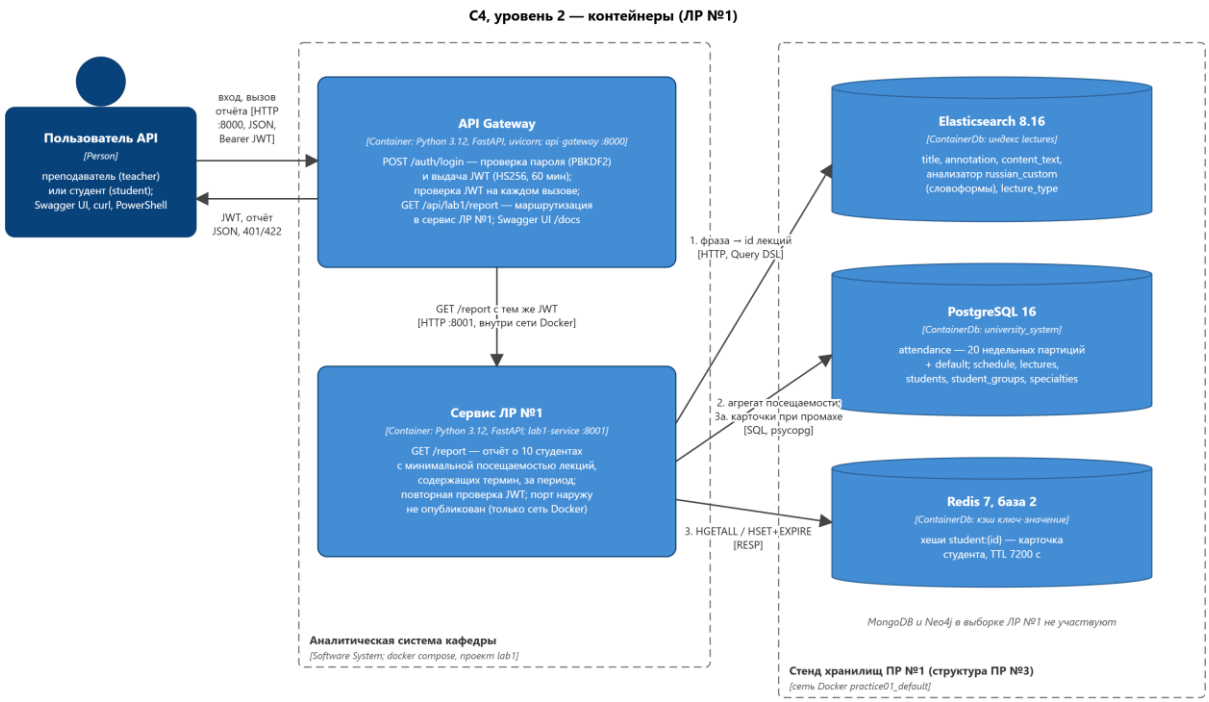


Рисунок 2 — C4, уровень 2: контейнеры

Таблица 2 — Контейнеры, участвующие в работе

Контейнер	**Порт**	**Назначение**
`api-gateway`	8000 (опубликован)	вход `POST /auth/login`, проверка JWT, маршрутизация `/api/lab1/*`, Swagger UI `/docs`
`lab1-service`	8001 (только сеть Docker)	метод `GET /report`: выборка из трёх хранилищ, повторная проверка JWT, трассировка шагов
`practice01-elasticsearch`	9200	индекс `lectures` — тексты занятий
`practice01-postgres`	5432 в сети (5433 на хосте)	база `university_system` — журнал посещаемости, расписание, студенты
`practice01-redis`	6379, база 2	кэш карточек студентов `student:{id}`

3.2. Диаграммы потоков данных (нотация Йордана–Коуда)

В нотации Йордана–Коуда [4] процессы изображаются окружностями с номером, внешние сущности — прямоугольниками, хранилища данных — парой параллельных линий с идентификатором, потоки — подписанными стрелками. Контекстная диаграмма (рисунок 3) показывает систему одним процессом 0 и её обмен с внешними сущностями; диаграмма уровня 1 (рисунок 4) раскрывает процесс 0.



Рисунок 3 — DFD, контекстная диаграмма

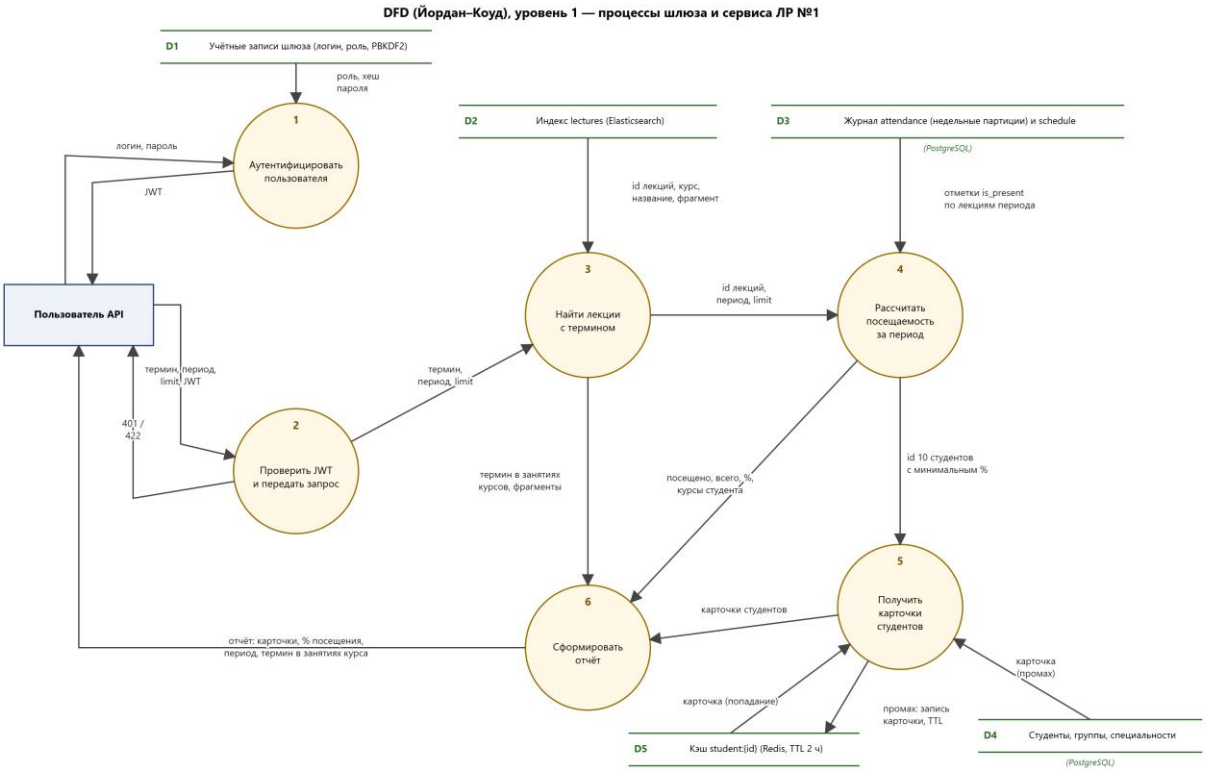


Рисунок 4 — DFD, уровень 1: процессы шлюза и сервиса ЛР №1

Таблица 3 — Процессы DFD уровня 1

Процесс	**Где выполняется**	**Вход → выход**
1. Аутентифицировать пользователя	API Gateway	логин, пароль + D1 → JWT
2. Проверить JWT и передать запрос	API Gateway	запрос с JWT → параметры для сервиса или 401/422
3. Найти лекции с термином	сервис ЛР №1 + D2	термин → id лекций, курсы, фрагменты текста
4. Рассчитать посещаемость за период	сервис ЛР №1 + D3	id лекций, период → 10 студентов с минимальным процентом
5. Получить карточки студентов	сервис ЛР №1 + D5, D4	id студентов → полные карточки (кэш, при промахе — PostgreSQL)
6. Сформировать отчёт	сервис ЛР №1	результаты 3–5 → JSON-отчёт

3.3. API шлюза и аутентификация

Таблица 4 — Методы API Gateway

Метод	Доступ	Ответ
`POST /auth/login` (форма username, password)	открыт	`access_token`, `token_type`, `expires_in`; 401 при неверной паре
`GET /auth/me`	JWT	полезная нагрузка токена
`GET /api/lab1/report?term&date_from&date_to&limit&trace`	JWT	отчёт ЛР №1; 401 без токена или с недействительным токеном, 422 при неверных параметрах
`GET /health`	открыт	состояние шлюза и сервиса ЛР №1
`GET /docs`	открыт	Swagger UI с кнопкой Authorize (OAuth2 password flow [7])

– пароли пользователей шлюза хранятся только как PBKDF2-SHA256 [8, 9] с солью и 200 000 итераций (переменная `GATEWAY_USERS` в `.env`); сравнение хеша — за постоянное время (`hmac.compare_digest`);

– токен — JWT [6], подпись HS256 общим секретом `JWT_SECRET`; полезная нагрузка: `sub` (логин), `role`, `iss` = papo-gateway, `iat`, `exp` (60 минут); при проверке обязательны `exp` и `sub`, сверяется издатель;

– шлюз проверяет токен на каждом вызове и передаёт сервису тот же заголовок `Authorization`; сервис проверяет подпись повторно, поэтому даже изнутри сети Docker его нельзя вызвать без действительного токена (эшелонированная защита);

– порт сервиса ЛР №1 на хост не публикуется: снаружи доступен только шлюз.

4. Структура хранения, задействованная в выборке

Данные — структура практической работы №3: 77 студентов в 4 группах двух специальностей (09.03.04 «Программная инженерия» и 38.03.05 «Бизнес-информатика»), 12 курсов, 296 занятий, из них 152 лекции, 592 занятия в расписании весеннего семестра 2026 года и 10 120 отметок посещаемости. Идентификаторы — UUID v5, одинаковые во всех хранилищах, поэтому результат одного шага без преобразований служит входом следующего.

Таблица 5 — Задействованные структуры хранения

Хранилище	**Структура**	**Поля, используемые в ЛР №1**
Elasticsearch, индекс `lectures` (296 документов)	документ на занятие; текстовые поля с анализатором `russian_custom` (стемминг, стоп-слова), ключевые поля для фильтров	`title`, `annotation`, `content_text` — поиск; `lecture_type` — фильтр; `lecture_id`, `course_name`, `specialty_code` — результат
PostgreSQL, `attendance`	журнал отметок, `PARTITION BY RANGE (week_start_date)`: 20 недельных партиций `attendance_w2026_02_09` … `attendance_w2026_06_22` и `attendance_default`	`schedule_id`, `student_id`, `week_start_date`, `is_present`
PostgreSQL, `schedule`	занятие группы на дату; индексы по `lecture_id`, `scheduled_date`	`id`, `lecture_id`, `scheduled_date`
PostgreSQL, `lectures`, `lecture_courses`	занятия и курсы	`lectures.course_id`, `lecture_courses.name` — курсы студента с термином
PostgreSQL, `students`, `student_groups`, `specialties`	карточка студента с группой и специальностью (источник при промахе кэша)	все поля карточки
Redis, база 2	хеш `student:{id}`, TTL 7200 с	`id`, `student_card_number`, ФИО, `group`, `specialty_code`, `specialty`, `email`, `phone`, `enrollment_date`, `status`

MongoDB (материалы занятий как документы) и Neo4j (иерархия университета и связи «группа — студент — курс») в выборке не участвуют: тексты лекций для поиска индексированы в Elasticsearch, а состав групп и факт посещения однозначно определяются журналом посещаемости, в котором есть и студент, и занятие.

5. Обоснование способа выборки

Запрос объединяет три разные по природе операции: полнотекстовый поиск по текстам занятий, точный агрегат по большому журналу за период и чтение карточек нескольких студентов. Каждая выполняется хранилищем, модель которого подходит для неё лучше всего [15], а порядок шагов выбран так, чтобы каждый следующий шаг работал с наименьшим объёмом данных.

5.1. Шаг 1 — Elasticsearch: лекции с термином

– термин нужно найти в названии, аннотации и тексте лекции во всех словоформах («очереди сообщений» находит «Очередь сообщений»);

анализатор `russian_custom` даёт стемминг [11], а запрос `multi_match` с `type: phrase` требует слова фразы подряд и в том же порядке;

– в PostgreSQL это потребовало бы `ILIKE` по всем текстам без учёта морфологии и без индекса или отдельной настройки `tsvector`; в Elasticsearch инвертированный индекс уже построен в практической работе №3;

– фильтр `lecture_type = lecture` оставляет только лекции, как требует задание; веса полей (`title^3`, `annotation^2`) нужны для подсветки и порядка примеров, на состав выборки не влияют;

– подсветка (`highlight`) возвращает фрагмент текста с термином — это поле «термин в занятиях курса»;

– выдача читается постранично через `search_after` по `lecture_id` (страница 200), поэтому при любом числе совпадений ни одна лекция не теряется; `track_total_hits` позволяет сверить полноту.

Результат шага — множество идентификаторов лекций (для термина «масштабирование» — 17 из 152 лекций в двух курсах). Оно сужает журнал посещаемости до нужных занятий ещё до обращения к PostgreSQL.

5.2. Шаг 2 — PostgreSQL: посещаемость за период

– журнал посещаемости — транзакционные данные, его единственная полная и точная копия хранится в PostgreSQL; процент посещения должен считаться точно по всем отметкам, поэтому агрегат выполняется там же, где лежат данные (`GROUP BY` на стороне СУБД, в сервис возвращается только 10 строк);

– журнал секционирован по неделям; условие `week_start_date BETWEEN <понедельник недели date_from> AND date_to` позволяет планировщику отсечь партиции вне периода (partition pruning) [12, 13]: для периода 2026-02-09 … 2026-04-19 читаются 10

недельных партиций из 20 и не читается партиция по умолчанию (рисунок 11);

– точная граница периода задаётся условием на `schedule.scheduled_date`, а занятия — массивом идентификаторов из шага 1 (`lecture_id = ANY(...)`, индекс `idx_schedule_lecture`);

– процент посещения студента = посещённые занятия-лекции с термином / все такие занятия его группы в периоде × 100, округление до 0,1; сортировка по возрастанию процента, при равенстве — по числу занятий (больше пропусков при той же доле — выше), затем по id для однозначности; `LIMIT 10`;

– для каждого студента собирается список курсов, в лекциях которых встретился термин (`array_agg(DISTINCT lc.name)`).

```
SELECT a.student_id::text AS student_id,
       count(*)                                  AS total,
       count(*) FILTER (WHERE a.is_present)      AS attended,
       round(100.0 * count(*) FILTER (WHERE a.is_present) / count(*), 1) AS attendance_pct,
       array_agg(DISTINCT lc.name ORDER BY lc.name) AS term_courses
FROM attendance a
JOIN schedule s         ON s.id = a.schedule_id
JOIN lectures l         ON l.id = s.lecture_id
JOIN lecture_courses lc ON lc.id = l.course_id
WHERE s.lecture_id = ANY(%(ids)s::uuid[])
  AND a.week_start_date BETWEEN %(week_from)s AND %(date_to)s       -- отсечение партиций
      журнала
  AND s.scheduled_date  BETWEEN %(date_from)s AND %(date_to)s
GROUP BY a.student_id
ORDER BY attendance_pct, total DESC, a.student_id
LIMIT %(limit)s
```

5.3. Шаг 3 — Redis: карточки студентов

– для отчёта нужна полная карточка только 10 студентов; карточки читаются часто (отчёты, расписание, журналы) и меняются редко, поэтому хранятся в Redis как хеши `student:{id}` (структура практической работы №3) и читаются одним конвейером `HGETALL` за одно обращение к серверу;

– стратегия cache-aside [14]: если ключа нет или в нём не хватает поля, недостающие карточки читаются из PostgreSQL одним запросом (соединение `students`, `student_groups` и `specialties`) и записываются в Redis с TTL 7200 с; устаревание кэша ограничено

временем жизни, а в лабораторных работах №4–6 изменения будут приходить через CDC;

– отказ от кэша не меняет результат, а только время ответа: протокол показывает оба варианта (рисунок 12).

5.4. Порядок шагов

Сначала выполняется самый избирательный шаг: поиск сокращает 152 лекции до 17, после чего PostgreSQL агрегирует 462 отметки из 10 партиций вместо 10 120 отметок журнала, а Redis отдаёт 10 карточек. Между хранилищами передаются только идентификаторы, данные не копируются целиком в сервис. Каждый шаг записывается в трассировку ответа (`trace=true`) и в журнал контейнера: хранилище, действие, результат и время.

6. Реализация и развёртывание

Таблица 6 — Состав проекта

Файл (каталог practice-db\lab1)	**Содержимое**
`app\common\security.py`	PBKDF2, выпуск и проверка JWT, зависимость `current_user`
`app\common\web.py`	явная кодировка UTF-8 в ответах JSON
`app\gateway\main.py`	API Gateway: вход, проверка токена, маршрутизация в сервис ЛР №1
`app\lab1\main.py`, `app\lab1\query.py`	сервис ЛР №1: метод `/report`, шаги выборки, трассировка
`Dockerfile`, `requirements.txt`	образ `papo-lab:1.0` (python:3.12-slim, запуск от `nobody`)
`docker-compose.yml`, `.env`	контейнеры `api-gateway` и `lab1-service`, ограничение памяти 256 МБ, проверки здоровья, внешняя сеть стенда; секрет JWT и пользователи шлюза
`queries\step1_es.json`	запрос шага 1 для независимой проверки в Elasticsearch
`tests\smoke.py`, `tests\verify.py`	быстрая проверка и независимая сверка с прямым SQL
`diagrams\make_diagrams.py`	исходники диаграмм C4 и DFD

Запуск: стенд практической работы №1 с данными практической работы №3 уже работает, затем в каталоге `lab1` выполняется `docker compose up -d --build`. Шлюз стартует после того, как сервис ЛР №1 пройдёт проверку здоровья; Swagger UI — `http://localhost:8000/docs`.

7. Протокол тестирования

Все проверки выполнены 17.10.2026 на рабочем компьютере. Команды на рисунках набраны в Windows PowerShell и выполнены по порядку; вывод под каждой командой — настоящий. Основной пример: термин «масштабирование», период 2026-02-09 … 2026-04-19 (первые 10 недель семестра).

Таблица 7 — Сводка протокола тестирования

№	Проверка	Ожидается	Получено	Рис.
1	Контейнеры запущены, сервис закрыт снаружи	оба контейнера healthy, порт 8001 недоступен	healthy, `TcpTestSucceeded = False`	5
2	Вызов без токена	401	401 Not authenticated	6
3	Вход с неверным паролем	401	401 «Неверный логин или пароль»	6
4	Вход teacher и student	JWT, срок 3600 с	токены выданы, `/auth/me` вернул claims	6
5	Отчёт с токеном	200	200, 5590 байт	6
6	Токен, подписанный чужим ключом	401	401 Signature verification failed	7
7	Токен с истёкшим сроком	401	401 «Срок действия токена истёк»	7
8	Прямой вызов сервиса в обход шлюза	401 без токена и с поддельным	401 в обоих случаях	7
9	Отчёт с трассировкой	4 шага, 10 студентов	Elasticsearch → PostgreSQL → Redis → сборка	8, 9
10	Шаг 1 напрямую в Elasticsearch	те же 17 лекций	17 лекций, 2 курса	10
11	Шаг 2 напрямую в PostgreSQL	24 занятия, 462 отметки, 10 партиций	совпадает; в плане 10 партиций	11
12	Шаг 3: промах и попадание кэша	3 промаха, затем 10 попаданий	7/3, затем 10/0; TTL 7200	12
13	Независимая сверка топ-10	совпадение построчно	совпадает (2 термина)	13
14	Неверный период, короткий термин	422	422 в обоих случаях	14
15	Фраза в другой словоформе	находится «Очередь сообщений»	найдена, отчёт из 3 строк	14
16	Термин не встречается	пустой отчёт без обращения к PostgreSQL и Redis	0 лекций, 2 шага	14
17	Журналы контейнеров	вход, маршрутизация, 4 шага	записи есть	15
18	Swagger UI: вход и вызов	Authorize → 200 с отчётом	выполнено	16–20

7.1. Развёртывание

Рисунок 5: оба контейнера в состоянии healthy, наружу опубликован только порт шлюза 8000; образ `papo-lab:1.0` общий; контейнеры ЛР №1 подключены к сети стенда вместе с хранилищами; `/health` шлюза сообщает, что сервис ЛР №1 доступен, а порт 8001 с хоста закрыт.

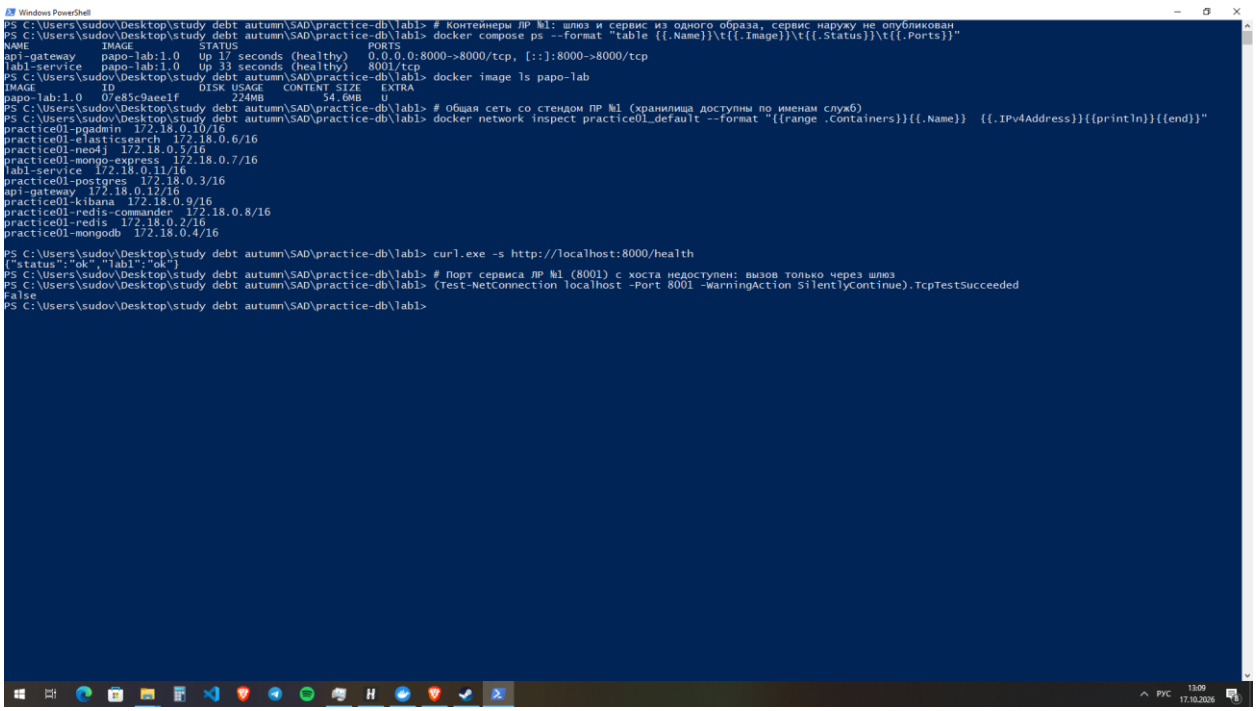


Рисунок 5 — Контейнеры ЛР №1, сеть стенда и закрытый порт сервиса

7.2. Аутентификация и защита методов

Рисунок 6: без токена и с неверным паролем — 401; с верным паролем шлюз выдаёт JWT, `/auth/me` показывает его полезную нагрузку (sub, role, iss, iat, exp), а вызов отчёта с токеном возвращает 200. Второй пользователь шлюза (роль student) также получает токен.

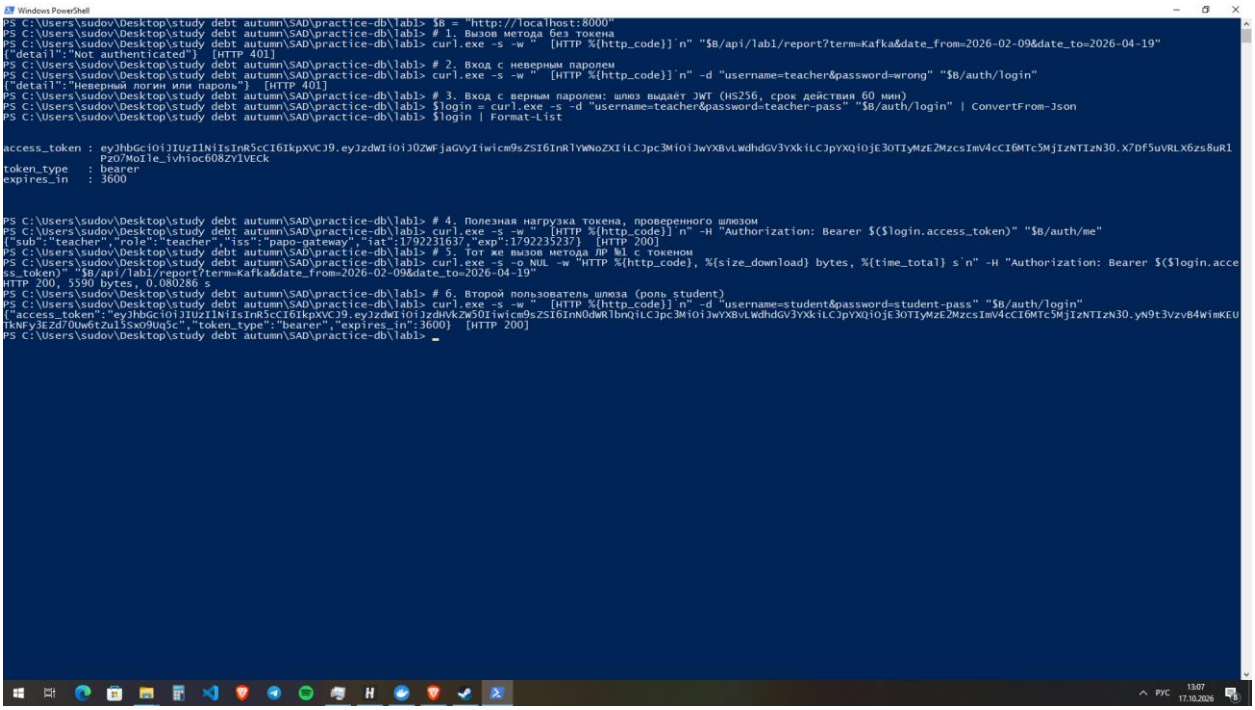


Рисунок 6 — Вход в шлюз и вызов метода с токеном и без него

Рисунок 7: токен с правильной структурой, но подписанный чужим ключом и с повышенной ролью, отклоняется — «Signature verification failed»; токен, подписанный верным ключом, но с истёкшим сроком, отклоняется — «Срок действия токена истёк». Прямой вызов сервиса ЛР №1 изнутри сети Docker в обход шлюза без токена и с поддельным токеном тоже получает 401: сервис проверяет подпись сам.

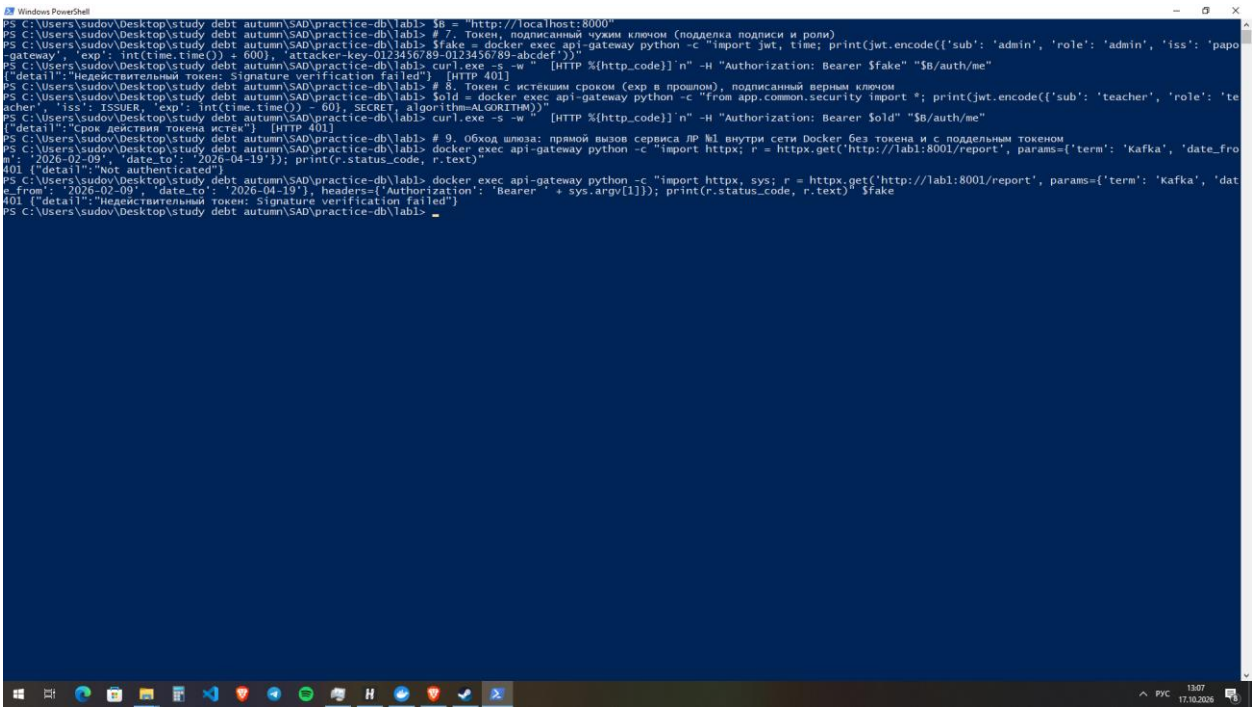


Рисунок 7 — Поддельный и просроченный токены, попытка обойти шлюз

7.3. Последовательность исполнения запроса

Рисунок 8: вызов `GET /api/lab1/report` с `trace=true`. Отчёт содержит термин, период, число найденных лекций, объём выборки (24 занятия, 462 отметки, 77 студентов) и пользователя, вызвавшего метод. Поле «термин в занятиях курсов» — курсы с числом лекций, где найден термин, и фрагменты текста с подсветкой. Ниже — трассировка исполнения по шагам.

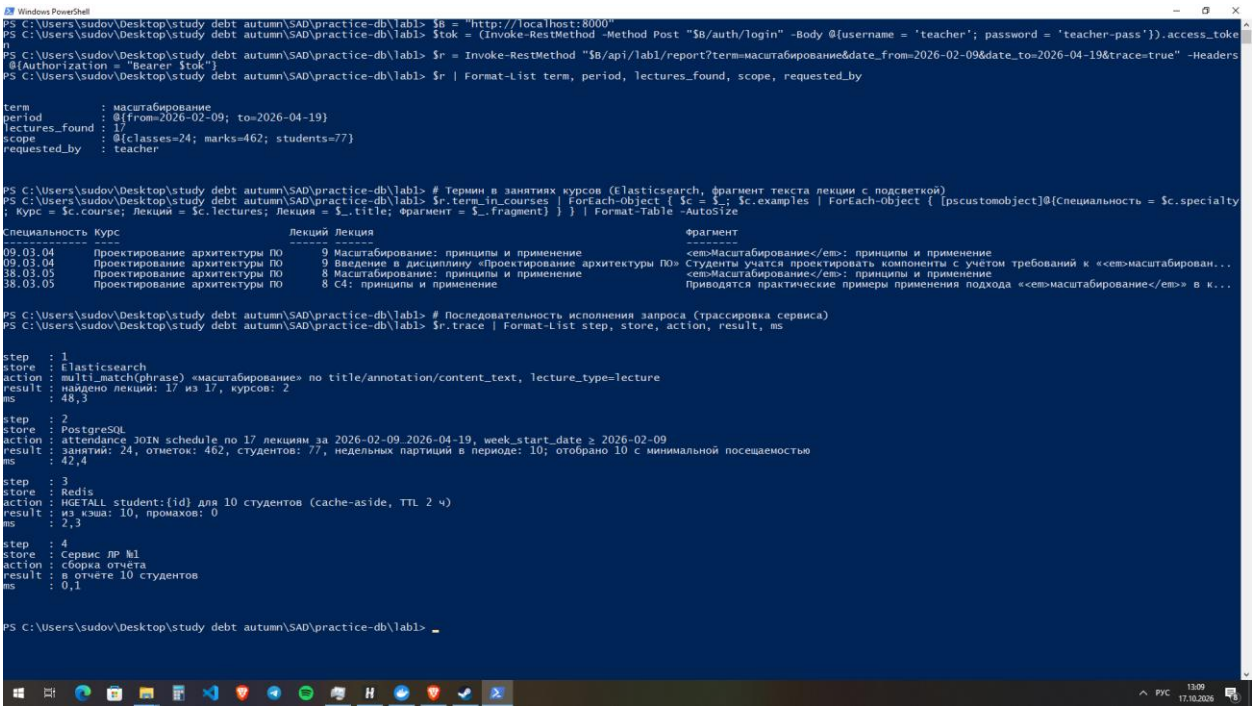


Рисунок 8 — Вызов отчёта: параметры, термин в курсах и шаги исполнения

Таблица 8 — Шаги исполнения (трассировка ответа, основной пример)

Шаг	Хранилище	Действие	Результат шага	мс
1	Elasticsearch	multi_match(phrase) «масштабирование» по title/annotation/content_text, lecture_type=lecture	найдено лекций: 17 из 17, курсов: 2	17,0
2	PostgreSQL	attendance JOIN schedule по 17 лекциям за 2026-02-09…2026-04-19, week_start_date ≥ 2026-02-09	занятий: 24, отметок: 462, студентов: 77, недельных партиций в периоде: 10; отобрано 10 с минимальной посещаемостью	41,1
3	Redis	HGETALL student:{id} для 10 студентов (cache-aside, TTL 2 ч)	из кэша: 10, промахов: 0	1,1
4	Сервис ЛР №1	сборка отчёта	в отчёте 10 студентов	0,1

Рисунок 9 и таблица 9: итоговый отчёт — 10 студентов с минимальным процентом посещения лекций с термином «масштабирование» за период. Для каждого — полная карточка, число посещённых лекций и всех лекций с

термином в периоде, процент, период и курсы, в лекциях которых встретился термин.

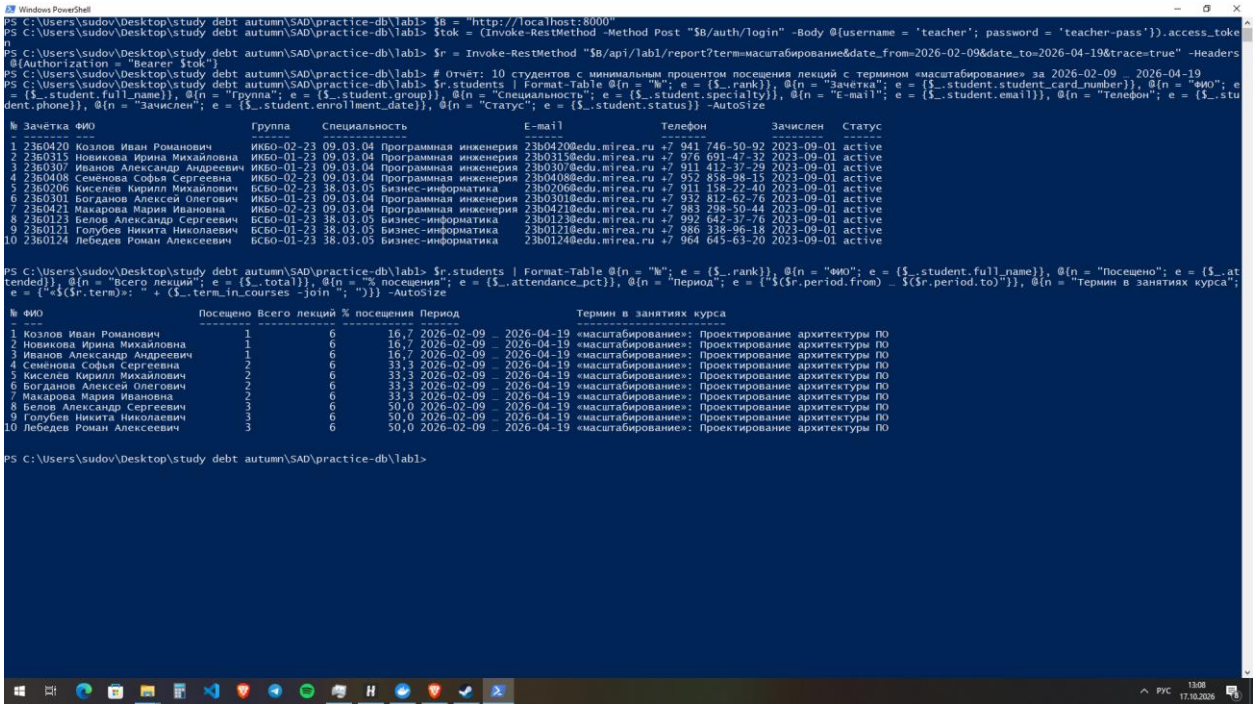


Рисунок 9 — Отчёт: 10 студентов с минимальной посещаемостью лекций с термином

Таблица 9 — Результат: термин «масштабирование», период 2026-02-09 … 2026-04-19

№	Студент	Группа, специальность	Контакты, зачисление	Лекции	%
1	Козлов Иван Романович зачётка 23Б0420, active	ИКБО-02-23 09.03.04 Программная инженерия	23b0420@edu.mirea.ru +7 941 746-50-92 зачислен 2023-09-01	1 из 6	16,7
2	Новикова Ирина Михайловна зачётка 23Б0315, active	ИКБО-01-23 09.03.04 Программная инженерия	23b0315@edu.mirea.ru +7 976 691-47-32 зачислен 2023-09-01	1 из 6	16,7
3	Иванов Александр Андреевич зачётка 23Б0307, active	ИКБО-01-23 09.03.04 Программная инженерия	23b0307@edu.mirea.ru +7 911 412-37-29 зачислен 2023-09-01	1 из 6	16,7
4	Семёнова Софья Сергеевна зачётка 23Б0408, active	ИКБО-02-23 09.03.04 Программная инженерия	23b0408@edu.mirea.ru +7 952 858-98-15 зачислен 2023-09-01	2 из 6	33,3
5	Киселёв Кирилл Михайлович зачётка 23Б0206, active	БСБО-02-23 38.03.05 Бизнес-информатика	23b0206@edu.mirea.ru +7 911 158-22-40 зачислен 2023-09-01	2 из 6	33,3
6	Богданов Алексей Олегович зачётка 23Б0301, active	ИКБО-01-23 09.03.04 Программная инженерия	23b0301@edu.mirea.ru +7 932 812-62-76 зачислен 2023-09-01	2 из 6	33,3
7	Макарова Мария Ивановна зачётка 23Б0421, active	ИКБО-02-23 09.03.04 Программная инженерия	23b0421@edu.mirea.ru +7 983 298-50-44 зачислен 2023-09-01	2 из 6	33,3
8	Белов Александр Сергеевич зачётка 23Б0123, active	БСБО-01-23 38.03.05 Бизнес-информатика	23b0123@edu.mirea.ru +7 992 642-37-76 зачислен 2023-09-01	3 из 6	50,0
9	Голубев Никита Николаевич зачётка 23Б0121, active	БСБО-01-23 38.03.05 Бизнес-информатика	23b0121@edu.mirea.ru +7 986 338-96-18 зачислен 2023-09-01	3 из 6	50,0

10	Лебедев Роман Алексеевич зачётка 23Б0124, active	БСБО-01-23 38.03.05 Бизнес-информатика	23b0124@edu.mirea.ru +7 964 645-63-20 зачислен 2023-09-01	3 из 6	50,0

Статус всех студентов — active; термин найден в курсе «Проектирование архитектуры ПО» обеих специальностей (9 лекций у 09.03.04 и 8 лекций у 38.03.05). У первых трёх студентов посещена 1 лекция из 6 (16,7 %), у последних в отчёте — 3 из 6 (50 %).

7.4. Проверка каждого шага отдельно

Рисунок 10: запрос шага 1, выполненный напрямую в Elasticsearch, возвращает те же 17 лекций: названия «Масштабирование: принципы и применение» и лекции, где термин встречается в аннотации или тексте (C4, монолит, микросервис, API gateway, введение в дисциплину).

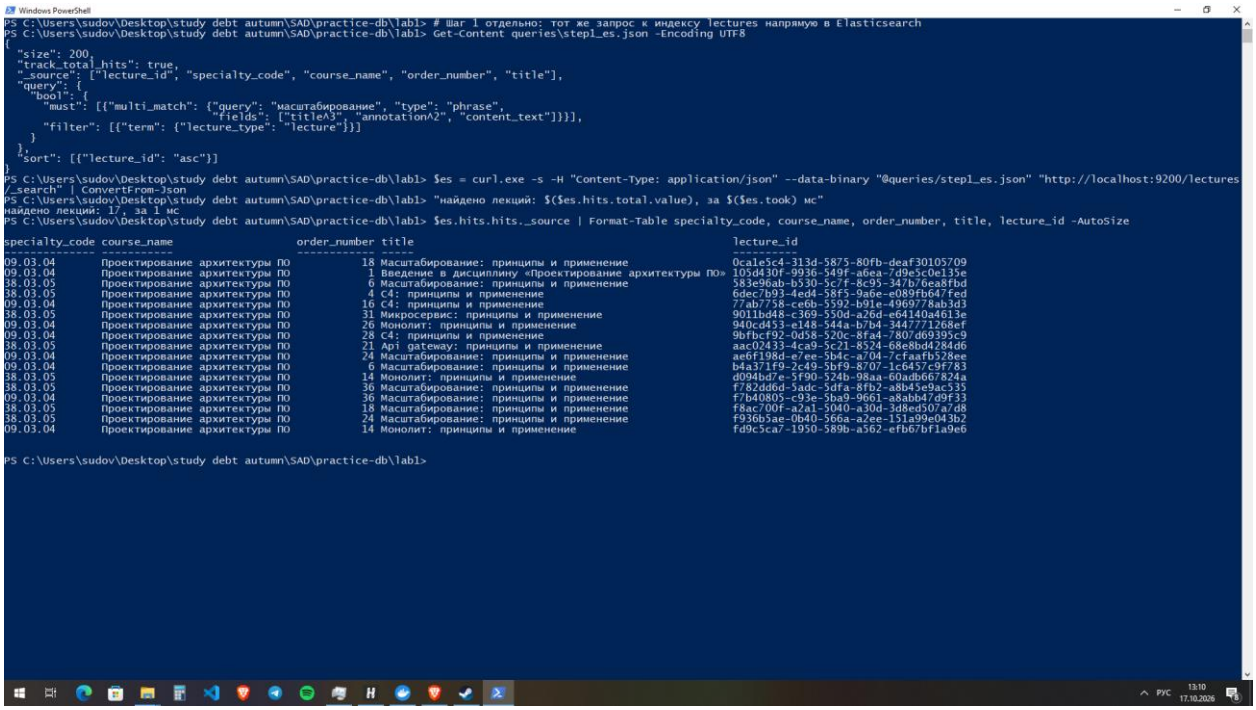


Рисунок 10 — Шаг 1 напрямую: запрос к индексу lectures

Рисунок 11: шаг 2 в psql с теми же идентификаторами лекций и границами периода даёт те же 24 занятия, 462 отметки и 77 студентов, а план запроса показывает чтение только партиций `attendance_w2026_02_09` … `attendance_w2026_04_13` (10 из 20, без партиции по умолчанию). При текущем объёме (592 строки расписания, по 600–700 отметок в партиции) планировщик выбирает последовательное чтение и хеш-соединение; индексы

`idx_schedule_lecture` и `uq_attendance_schedule_student` используются при росте объёма.

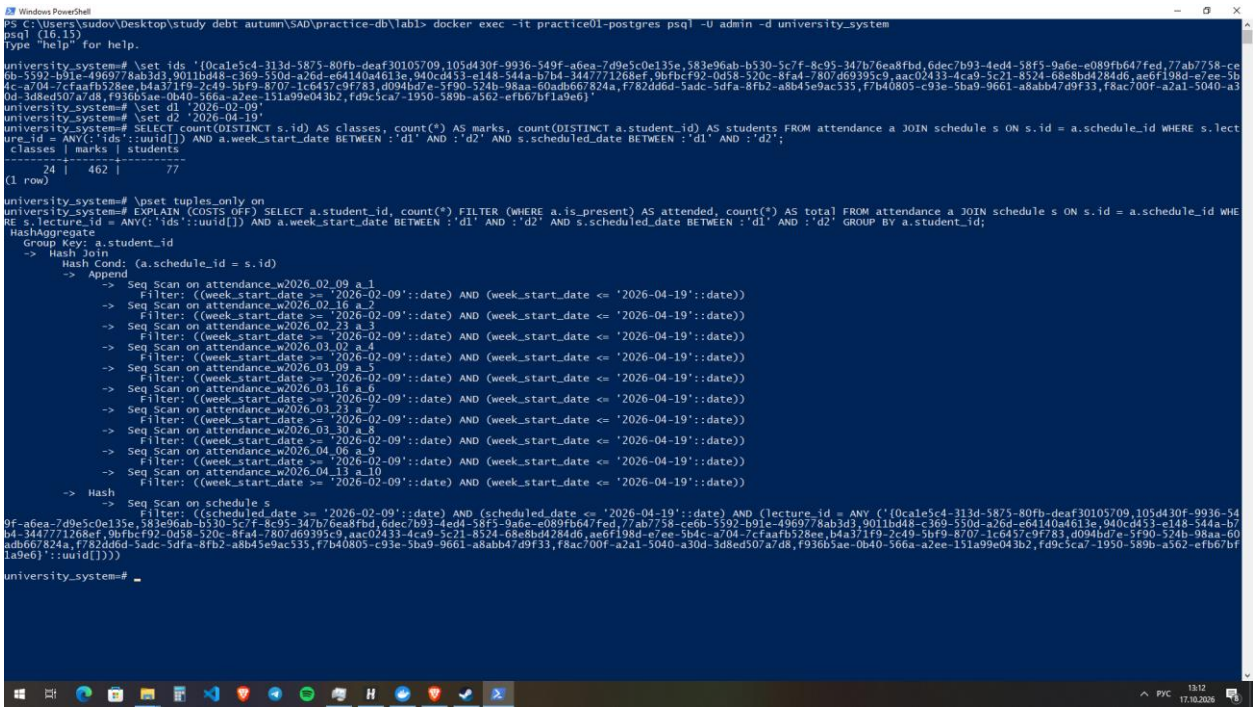


Рисунок 11 — Шаг 2 напрямую: объём выборки и отсечение партиций в плане запроса

Рисунок 12: карточки трёх первых студентов отчёта удалены из Redis (как после истечения TTL). Первый вызов: 7 карточек из кэша, 3 промаха — они прочитаны из PostgreSQL и записаны обратно. Повторный вызов: все 10 карточек из кэша, время шага сократилось с 19,5 до 1,3 мс. Восстановленная карточка содержит все 12 полей, TTL — 7200 с.

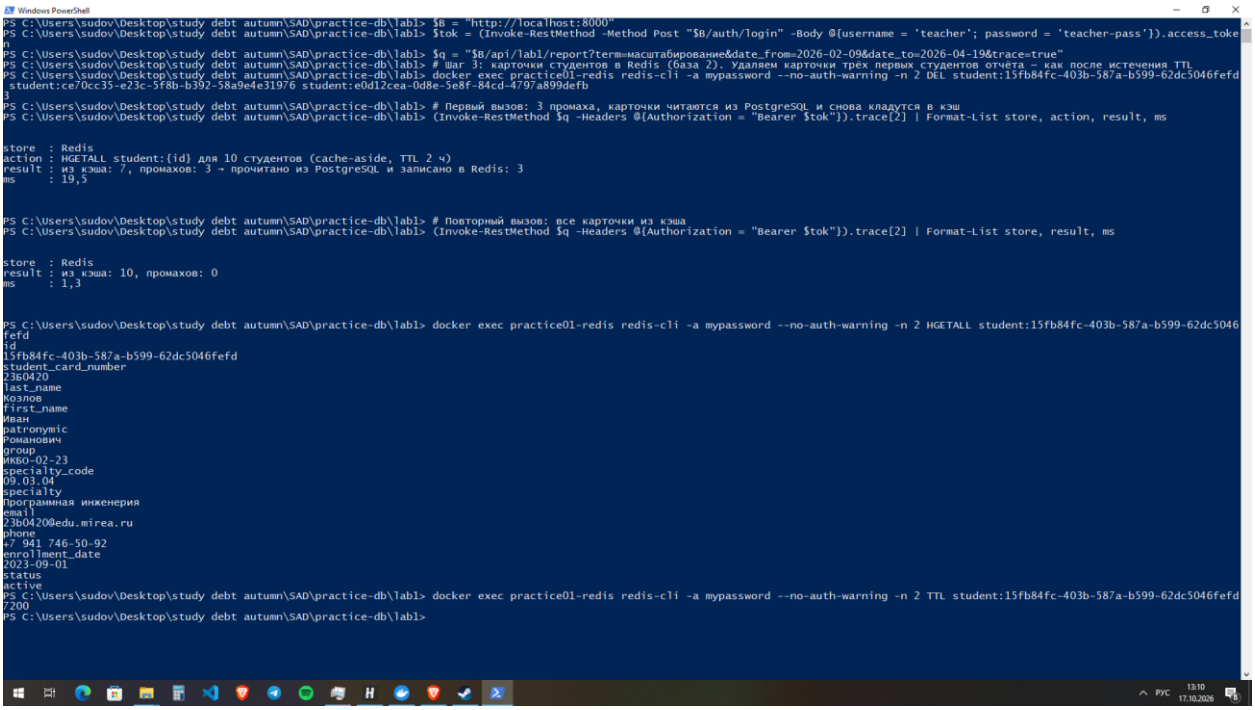


Рисунок 12 — Шаг 3: промах и попадание кэша карточек студентов

7.5. Независимая сверка результата

Скрипт `tests\verify.py` берёт из трассировки идентификаторы лекций шага 1 и пересчитывает посещаемость другим SQL-запросом напрямую в PostgreSQL — без условия на партиции, без кэша и с `CASE` вместо `FILTER`. Для термина «масштабирование» и для фразы «очередь сообщений» (другой период) все 10 строк совпали по зачётке, числу посещённых и всех лекций и проценту (рисунок 13).

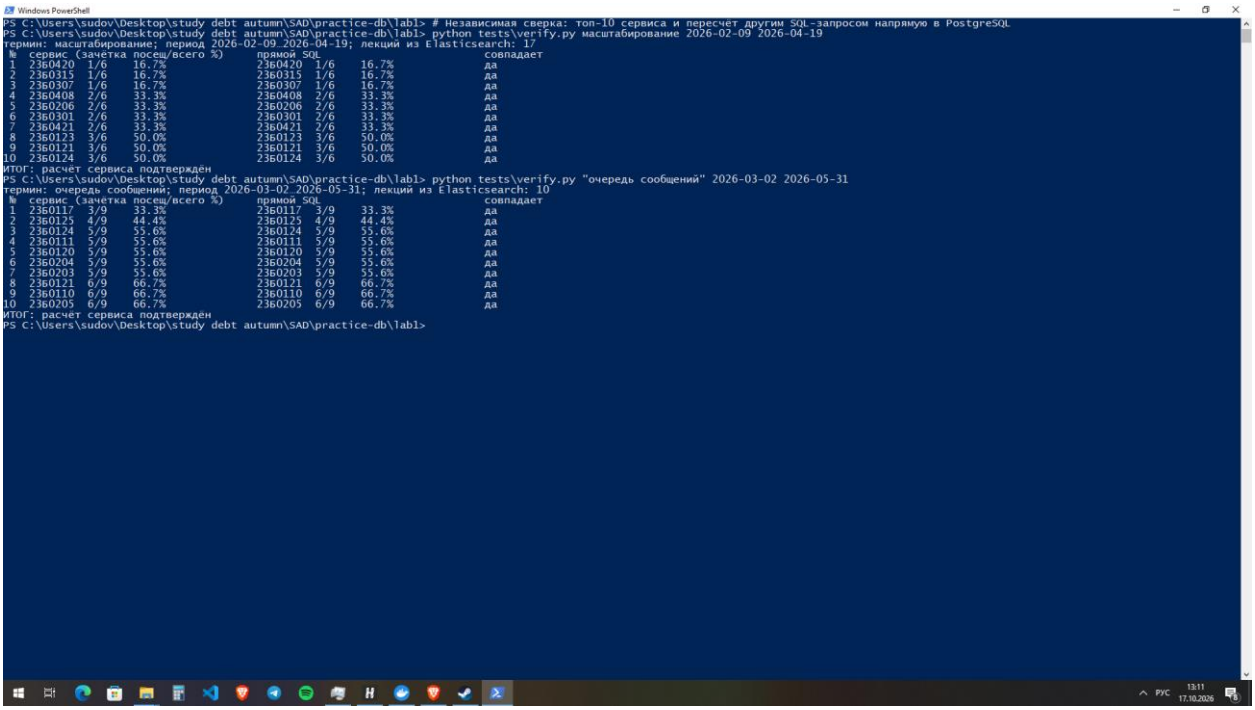


Рисунок 13 — Независимая сверка топ-10 сервиса с прямым SQL-запросом

7.6. Граничные случаи

Рисунок 14: начало периода позже конца — 422 от сервиса; термин из одного символа — 422 от шлюза (проверка параметров до обращения к сервису). Фраза «очереди сообщений» находит лекции «Очередь сообщений: принципы и применение» — работает русский анализатор; `limit=3` ограничивает отчёт тремя строками. Фраза «горизонтальное масштабирование» не встречается ни в одной лекции: отчёт пустой, выполнены только поиск и сборка, PostgreSQL и Redis не вызываются.

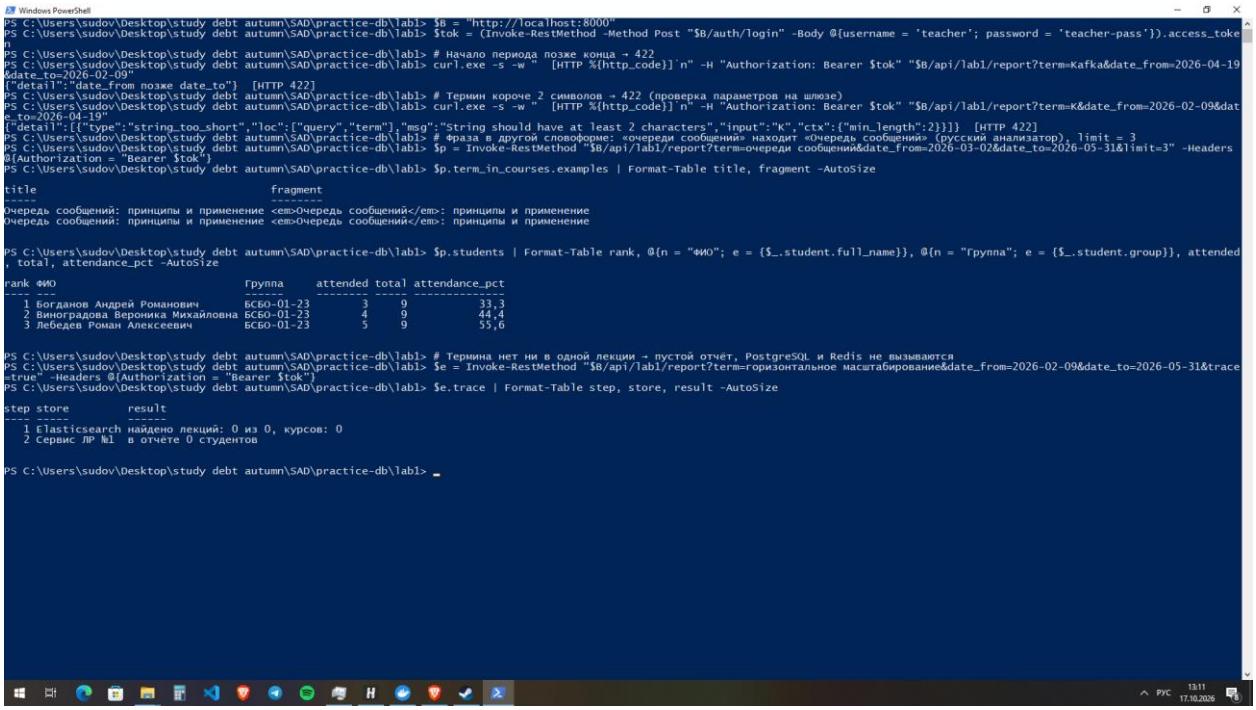


Рисунок 14 — Граничные случаи: ошибки параметров, словоформы, пустой результат

7.7. Журналы контейнеров

Рисунок 15: в журнале шлюза — выдача токена и маршрутизация вызова с пользователем и итоговым кодом, в журнале сервиса — четыре шага выборки с результатами и временем.

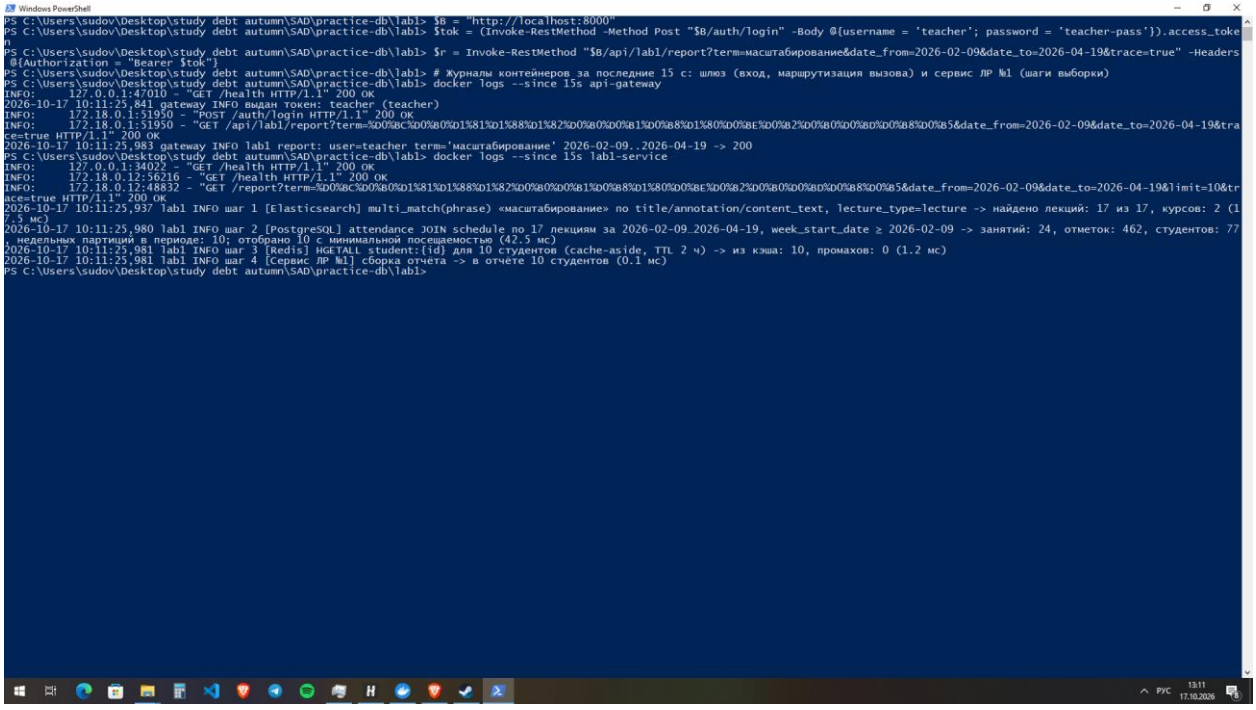


Рисунок 15 — Журналы api-gateway и lab1-service за один вызов

7.8. Вызов через Swagger UI

Рисунки 16–20: документация шлюза в Swagger UI, вход по кнопке Authorize (OAuth2 password flow — Swagger UI сам вызывает `/auth/login` и подставляет токен), заполнение параметров метода и ответ 200 с отчётом, заголовком `content-type: application/json; charset=utf-8` и карточкой первого студента.

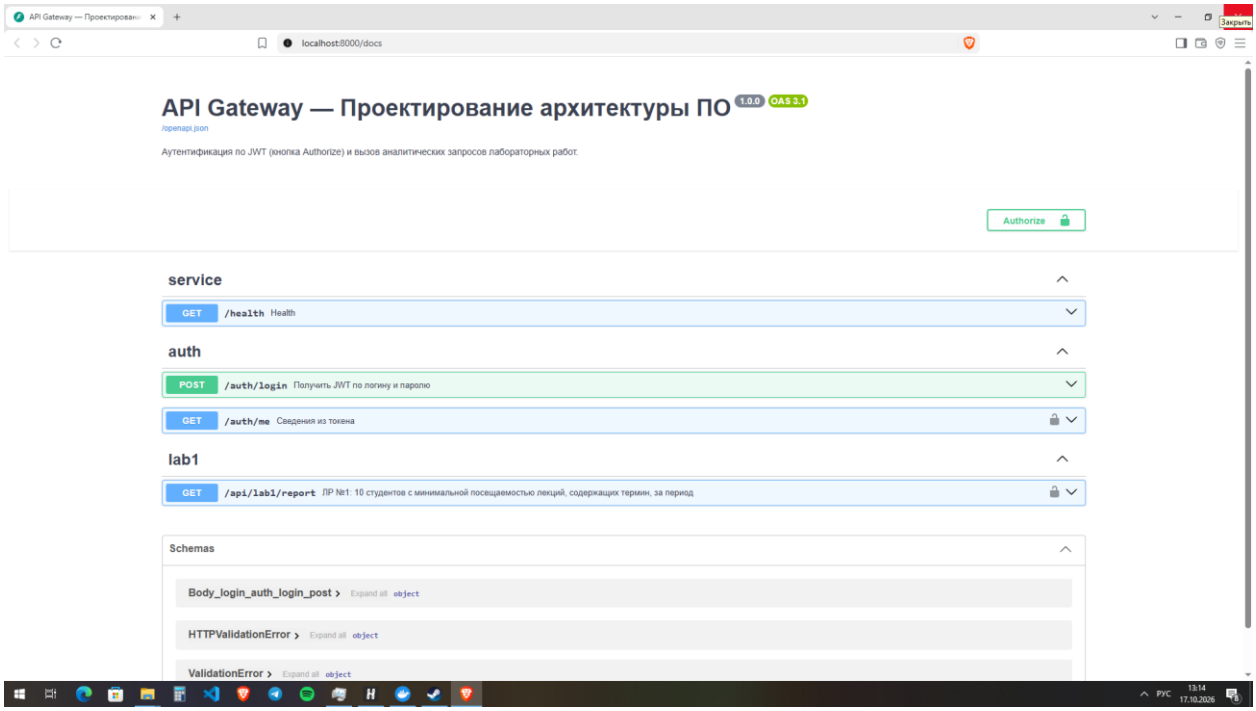


Рисунок 16 — Swagger UI шлюза: методы и кнопка Authorize

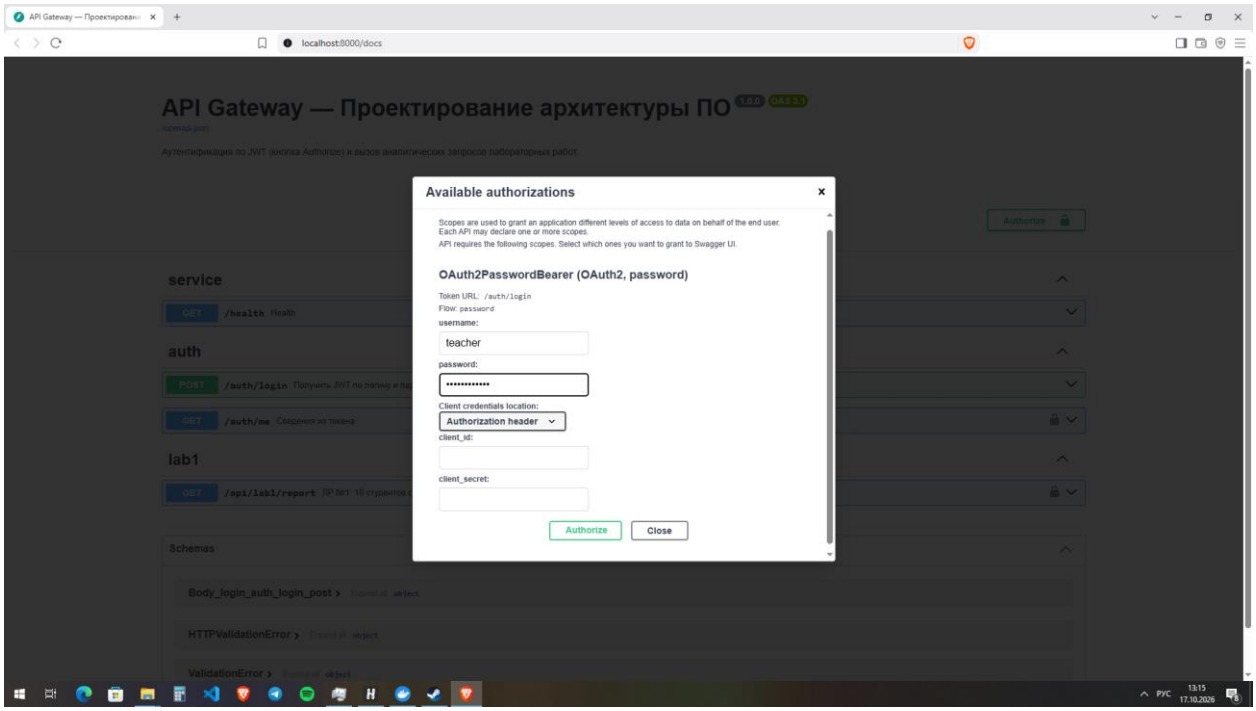


Рисунок 17 — Swagger UI: вход пользователя teacher

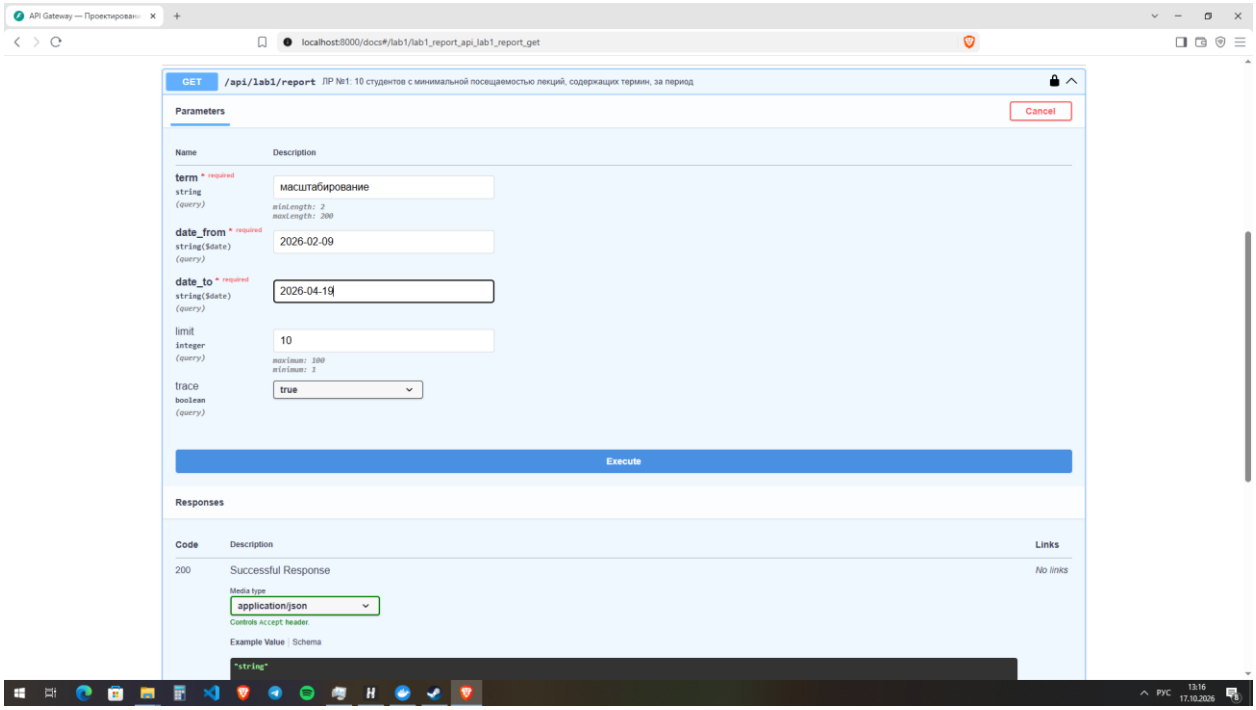


Рисунок 18 — Swagger UI: параметры метода /api/lab1/report (замок закрыт — токен передаётся)

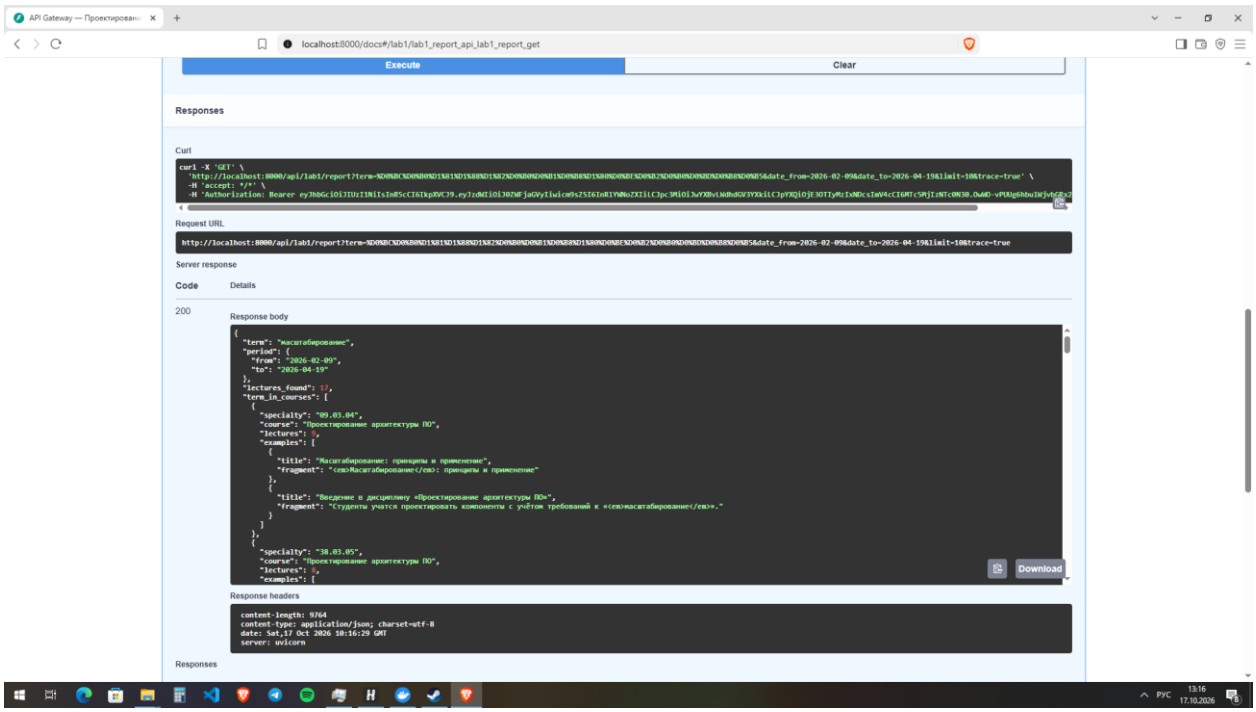


Рисунок 19 — Swagger UI: запрос curl, адрес и начало ответа

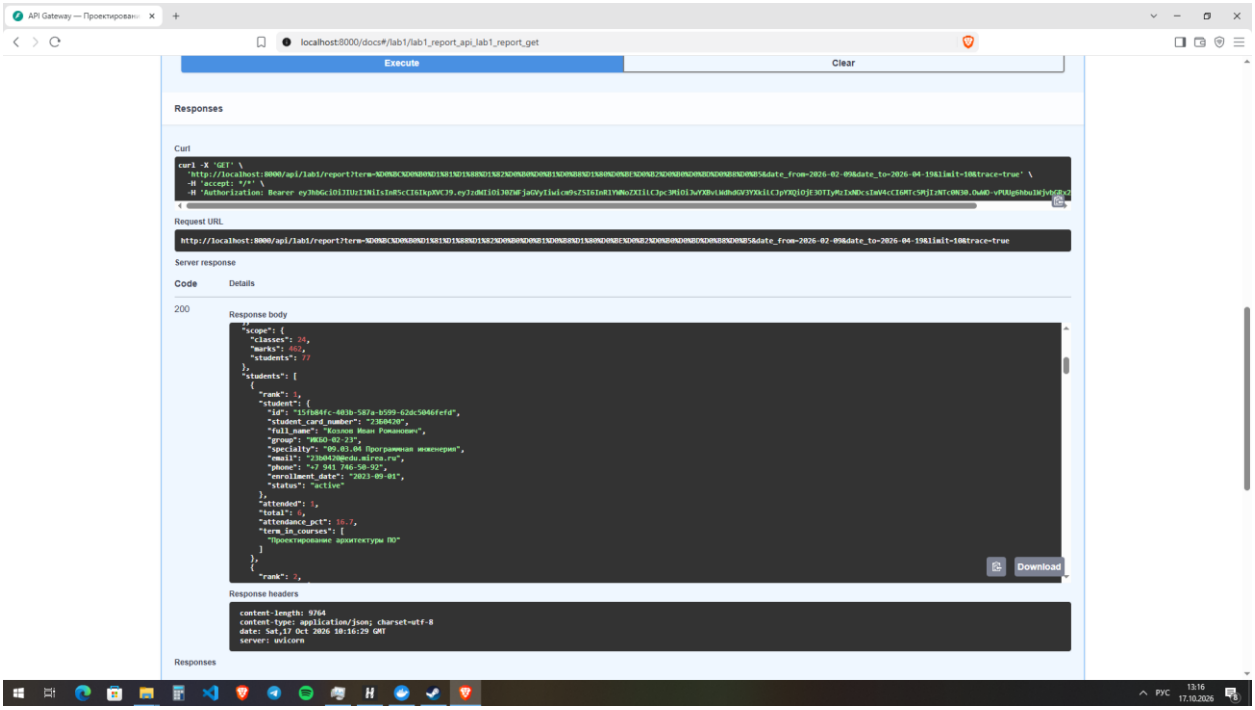


Рисунок 20 — Swagger UI: карточка студента в ответе

8. Выводы

Реализован аналитический запрос ЛР №1 — 10 студентов с минимальным процентом посещения лекций, содержащих заданный термин или фразу, за период — в виде отдельного сервиса в контейнере Docker. Вызов выполняется только через API Gateway с аутентификацией по JWT; сервис повторно проверяет подпись токена и наружу не опубликован.

Выборка распределена по хранилищам структуры практической работы №3 по их сильным сторонам: Elasticsearch находит лекции по фразе с учётом морфологии и даёт фрагменты текста, PostgreSQL точно агрегирует журнал посещаемости, читая только партиции периода, Redis отдаёт карточки студентов из кэша с восполнением из PostgreSQL. Протокол из 18 проверок подтвердил защиту методов, корректность каждого шага, совпадение результата с независимым пересчётом и работу кэша. Архитектура описана диаграммами C4 (контекст, контейнеры) и DFD в нотации Йордана–Коуда (контекстная и уровень 1).

Список использованных источников

1. Задание на практические и лабораторные работы по дисциплине «Проектирование архитектуры ПО». — Москва, 2026. — Текст : электронный.

2. Практическая работа №3. Проектирование логической и физической структуры данных : методические указания. — 2026. — 34 с. — Текст : электронный.

3. Brown S. The C4 model for visualising software architecture. — URL: https://c4model.com (дата обращения: 17.10.2026). — Текст : электронный.

4. Yourdon E. Modern Structured Analysis / E. Yourdon. — Englewood Cliffs : Prentice Hall, 1989. — 672 p. — Текст : непосредственный.

5. Ричардсон К. Микросервисы. Паттерны разработки и рефакторинга / К. Ричардсон. — Санкт-Петербург : Питер, 2019. — 544 с. — Текст : непосредственный.

6. Jones M., Bradley J., Sakimura N. JSON Web Token (JWT) : RFC 7519. — IETF, 2015. — URL: https://www.rfc-editor.org/rfc/rfc7519 (дата обращения: 17.10.2026). — Текст : электронный.

7. Hardt D. The OAuth 2.0 Authorization Framework : RFC 6749. — IETF, 2012. — URL: https://www.rfc-editor.org/rfc/rfc6749 (дата обращения: 17.10.2026). — Текст : электронный.

8. Moriarty K., Kaliski B., Rusch A. PKCS #5: Password-Based Cryptography Specification Version 2.1 : RFC 8018. — IETF, 2017. — URL: https://www.rfc-editor.org/rfc/rfc8018 (дата обращения: 17.10.2026). — Текст : электронный.

9. OWASP Cheat Sheet Series. Password Storage Cheat Sheet. — URL: https://cheatsheetseries.owasp.org/cheatsheets/Password_Storage_Cheat_Sheet.html (дата обращения: 17.10.2026). — Текст : электронный.

10. FastAPI Documentation. — URL: https://fastapi.tiangolo.com/ (дата обращения: 17.10.2026). — Текст : электронный.

11. Elasticsearch Guide 8.16. — URL: https://www.elastic.co/guide/en/elasticsearch/reference/8.16/index.html (дата обращения: 17.10.2026). — Текст : электронный.

12. PostgreSQL 16 Documentation. — URL: https://www.postgresql.org/docs/16/index.html (дата обращения: 17.10.2026). — Текст : электронный.

13. PostgreSQL 16 Documentation. Table Partitioning. — URL: https://www.postgresql.org/docs/16/ddl-partitioning.html (дата обращения: 17.10.2026). — Текст : электронный.

14. Redis Documentation. — URL: https://redis.io/docs/latest/ (дата обращения: 17.10.2026). — Текст : электронный.

15. Клеппман М. Высоконагруженные приложения. Программирование, масштабирование, поддержка / М. Клеппман. — Санкт-Петербург : Питер, 2018. — 640 с. — Текст : непосредственный.

Приложение А. Сервис ЛР №1: app\lab1\query.py

```
"""ЛР №1. Отчёт о 10 студентах с минимальным процентом посещения лекций, содержащих заданный термин
или фразу, за период обучения.


Порядок выборки (каждый шаг попадает в трассировку ответа и в журнал сервиса):
  1. Elasticsearch, индекс lectures — лекции (lecture_type = lecture), в названии, аннотации или тексте
     которых встречается термин/фраза: multi_match type=phrase с русским анализатором (словоформы),
     постраничный проход search_after — без молчаливого усечения выдачи.
  2. PostgreSQL — посещаемость найденных лекций за период: attendance ⋈ schedule по id лекций,
     условие на week_start_date отсекает партиции журнала (partition pruning); агрегат на стороне СУБД,
     сортировка по проценту посещения, первые N.
  3. Redis, база №2 — полные карточки найденных студентов по ключам student:{id} (cache-aside);
     промахи читаются из PostgreSQL одним запросом и кладутся в кэш с TTL 2 ч.
"""
from __future__ import annotations


import logging
import os
import time
from datetime import date, timedelta


import httpx
import psycopg
import redis
from psycopg.rows import dict_row


log = logging.getLogger('lab1')
ES_URL = os.environ.get('ES_URL', 'http://elasticsearch:9200')
PG_DSN = os.environ.get('PG_DSN', 'postgresql://admin:secret@postgres:5432/university_system')
REDIS_URL = os.environ.get('REDIS_URL', 'redis://:mypassword@redis:6379/2')
CACHE_TTL = 7200
PAGE = 200
STUDENT_FIELDS = ['id', 'student_card_number', 'last_name', 'first_name', 'patronymic', 'group',
    'specialty_code',
                  'specialty', 'email', 'phone', 'enrollment_date', 'status']


_redis = redis.Redis.from_url(REDIS_URL, decode_responses=True)




class Trace:
    def __init__(self):
        self.steps = []


    def add(self, store: str, action: str, result: str, started: float, details=None):
        ms = round((time.perf_counter() - started) * 1000, 1)
        self.steps.append({'step': len(self.steps) + 1, 'store': store, 'action': action, 'result': result,
                           'ms': ms, **({'details': details} if details is not None else {})})
        log.info('шаг %d [%s] %s -> %s (%.1f мс)', len(self.steps), store, action, result, ms)




def find_lectures(term: str, trace: Trace) -> list[dict]:
    t0 = time.perf_counter()
    body = {
        'size': PAGE, 'track_total_hits': True,
        '_source': ['lecture_id', 'course_id', 'course_name', 'title', 'specialty_code'],
        'query': {'bool': {
            'must': [{'multi_match': {'query': term, 'type': 'phrase',
                                      'fields': ['title^3', 'annotation^2', 'content_text']}}],
            'filter': [{'term': {'lecture_type': 'lecture'}}]}},
        'highlight': {'fields': {'content_text': {'fragment_size': 120, 'number_of_fragments': 1}, 'title':
            {}}},
        'sort': [{'lecture_id': 'asc'}],
    }
    lectures, total = [], None
    with httpx.Client(base_url=ES_URL, timeout=10) as es:
        while True:
            resp = es.post('/lectures/_search', json=body)
            resp.raise_for_status()
            data = resp.json()['hits']
            total = data['total']['value'] if total is None else total
            for h in data['hits']:
                src = h['_source']
                hl = h.get('highlight', {})
                lectures.append({**src, 'highlight': (hl.get('title') or hl.get('content_text') or [''])[0]})
            if len(data['hits']) < PAGE:
                break
            body['search_after'] = data['hits'][-1]['sort']        # следующая страница
    courses = sorted({(x['specialty_code'], x['course_name']) for x in lectures})
```

```
    trace.add('Elasticsearch', f'multi_match(phrase) «{term}» по title/annotation/content_text,
        lecture_type=lecture',
              f'найдено лекций: {len(lectures)} из {total}, курсов: {len(courses)}', t0,
              {'courses': [{'specialty': s, 'course': c,
                            'lectures': sum(1 for x in lectures if x['course_name'] == c and
                                x['specialty_code'] == s)}
                           for s, c in courses],
               'lecture_ids': [x['lecture_id'] for x in lectures]})
    return lectures
```

```
ATTENDANCE_SQL = """
SELECT a.student_id::text AS student_id,
       count(*)                                  AS total,
       count(*) FILTER (WHERE a.is_present)      AS attended,
       round(100.0 * count(*) FILTER (WHERE a.is_present) / count(*), 1) AS attendance_pct,
       array_agg(DISTINCT lc.name ORDER BY lc.name) AS term_courses
FROM attendance a
JOIN schedule s         ON s.id = a.schedule_id
JOIN lectures l         ON l.id = s.lecture_id
JOIN lecture_courses lc ON lc.id = l.course_id
WHERE s.lecture_id = ANY(%(ids)s::uuid[])
  AND a.week_start_date BETWEEN %(week_from)s AND %(date_to)s       -- отсечение партиций журнала
  AND s.scheduled_date  BETWEEN %(date_from)s AND %(date_to)s
GROUP BY a.student_id
ORDER BY attendance_pct, total DESC, a.student_id
LIMIT %(limit)s
"""
SCOPE_SQL = """
SELECT count(DISTINCT s.id) AS classes, count(*) AS marks, count(DISTINCT a.student_id) AS students
FROM attendance a JOIN schedule s ON s.id = a.schedule_id
WHERE s.lecture_id = ANY(%(ids)s::uuid[])
  AND a.week_start_date BETWEEN %(week_from)s AND %(date_to)s
  AND s.scheduled_date  BETWEEN %(date_from)s AND %(date_to)s
"""
```

```
def attendance_top(ids: list[str], date_from: date, date_to: date, limit: int, trace: Trace) ->
    tuple[list[dict], dict]:
    t0 = time.perf_counter()
    params = {'ids': ids, 'date_from': date_from, 'date_to': date_to, 'limit': limit,
              'week_from': date_from - timedelta(days=date_from.weekday())}      # понедельник первой недели
    with psycopg.connect(PG_DSN, row_factory=dict_row) as conn:
        scope = conn.execute(SCOPE_SQL, params).fetchone()
        rows = conn.execute(ATTENDANCE_SQL, params).fetchall()
        names = [r['relname'] for r in conn.execute(
            "SELECT c.relname FROM pg_inherits i JOIN pg_class c ON c.oid = i.inhrelid "
            "WHERE i.inhparent = 'attendance'::regclass").fetchall()]
    # недельные партиции attendance_wГГГГ_ММ_ДД, попадающие в период (их и читает запрос после отсечения)
    weeks = [date.fromisoformat(n[12:].replace('_', '-')) for n in names if n.startswith('attendance_w')]
    parts = {'n': sum(1 for w in weeks if params['week_from'] <= w <= date_to)}
    trace.add('PostgreSQL', f'attendance JOIN schedule по {len(ids)} лекциям за {date_from}…{date_to}, '
                            f'week_start_date ≥ {params["week_from"]}',
              f'занятий: {scope["classes"]}, отметок: {scope["marks"]}, студентов: {scope["students"]}, '
              f'недельных партиций в периоде: {parts["n"]}; отобрано {len(rows)} с минимальной
                  посещаемостью', t0,
              [{k: (float(v) if k == 'attendance_pct' else v) for k, v in r.items()} for r in rows])
    return rows, scope
```

```
STUDENTS_SQL = """
SELECT st.id::text AS id, st.student_card_number, st.last_name, st.first_name, st.patronymic,
       g.name AS group, sp.code AS specialty_code, sp.name AS specialty, st.email, st.phone,
       st.enrollment_date::text AS enrollment_date, st.status
FROM students st JOIN student_groups g ON g.id = st.group_id JOIN specialties sp ON sp.id = g.specialty_id
WHERE st.id = ANY(%(ids)s::uuid[])
"""
```

```
def student_cards(ids: list[str], trace: Trace) -> dict[str, dict]:
    t0 = time.perf_counter()
    pipe = _redis.pipeline()
    for sid in ids:
        pipe.hgetall(f'student:{sid}')
    cached = dict(zip(ids, pipe.execute()))
    cards = {sid: c for sid, c in cached.items() if c and all(f in c for f in STUDENT_FIELDS)}
    misses = [sid for sid in ids if sid not in cards]
    if misses:
        with psycopg.connect(PG_DSN, row_factory=dict_row) as conn:
            rows = conn.execute(STUDENTS_SQL, {'ids': misses}).fetchall()
        pipe = _redis.pipeline()
        for r in rows:
            card = {k: '' if r[k] is None else str(r[k]) for k in STUDENT_FIELDS}
```

```
            cards[card['id']] = card
            pipe.hset(f'student:{card["id"]}', mapping=card)
            pipe.expire(f'student:{card["id"]}', CACHE_TTL)
        pipe.execute()
    trace.add('Redis', f'HGETALL student:{{id}} для {len(ids)} студентов (cache-aside, TTL {CACHE_TTL //
        3600} ч)',
              f'из кэша: {len(ids) - len(misses)}, промахов: {len(misses)}'
              + (f' → прочитано из PostgreSQL и записано в Redis: {len(misses)}' if misses else ''), t0)
    return cards



def build_report(term: str, date_from: date, date_to: date, limit: int = 10) -> dict:
    trace = Trace()
    lectures = find_lectures(term, trace)
    rows, scope = attendance_top([x['lecture_id'] for x in lectures], date_from, date_to, limit, trace) if
        lectures else ([], {})
    cards = student_cards([r['student_id'] for r in rows], trace) if rows else {}
    t0 = time.perf_counter()
    top = []
    for i, r in enumerate(rows, start=1):
        c = cards.get(r['student_id'], {})
        top.append({
            'rank': i,
            'student': {'id': r['student_id'], 'student_card_number': c.get('student_card_number'),
                        'full_name': ' '.join(filter(None, [c.get('last_name'), c.get('first_name'),
                            c.get('patronymic')])),
                        'group': c.get('group'), 'specialty': f'{c.get("specialty_code")}
                            {c.get("specialty")}',
                        'email': c.get('email'), 'phone': c.get('phone'), 'enrollment_date':
                            c.get('enrollment_date'),
                        'status': c.get('status')},
            'attended': r['attended'], 'total': r['total'], 'attendance_pct': float(r['attendance_pct']),
            'term_in_courses': r['term_courses'],
        })
    by_course = {}
    for x in lectures:
        by_course.setdefault((x['specialty_code'], x['course_name']), []).append({'title': x['title'],
            'fragment': x['highlight']})
    report = {
        'term': term,
        'period': {'from': date_from.isoformat(), 'to': date_to.isoformat()},
        'lectures_found': len(lectures),
        'term_in_courses': [{'specialty': s, 'course': c, 'lectures': len(v), 'examples': v[:2]}
                            for (s, c), v in sorted(by_course.items())],
        'scope': {k: int(v) for k, v in scope.items()} if scope else {},
        'students': top,
    }
    trace.add('Сервис ЛР №1', 'сборка отчёта', f'в отчёте {len(top)} студентов', t0)
    report['trace'] = trace.steps
    return report
```

Приложение Б. Сервис ЛР №1: app\lab1\main.py

```
"""Сервис ЛР №1 (отдельный контейнер). Доступен только из сети Docker; вызов — через API Gateway."""
import logging
from datetime import date


from fastapi import Depends, FastAPI, HTTPException, Query


from app.common.web import utf8_json
from app.common.security import current_user
from app.lab1.query import build_report


logging.basicConfig(level=logging.INFO, format='%(asctime)s %(name)s %(levelname)s %(message)s')
logging.getLogger('httpx').setLevel(logging.WARNING)      # без строки на каждый внутренний HTTP-запрос
app = FastAPI(title='Сервис ЛР №1: посещаемость лекций по термину', version='1.0.0')
app.middleware('http')(utf8_json)



@app.get('/health')
def health():
    return {'status': 'ok', 'service': 'lab1'}



@app.get('/report')
def report(term: str = Query(..., min_length=2, max_length=200, description='термин или фраза'),
           date_from: date = Query(..., description='начало периода, ГГГГ-ММ-ДД'),
           date_to: date = Query(..., description='конец периода, ГГГГ-ММ-ДД'),
           limit: int = Query(10, ge=1, le=100), trace: bool = Query(False, description='вернуть шаги
               выполнения'),
```

```
           user: dict = Depends(current_user)):
    if date_from > date_to:
        raise HTTPException(422, 'date_from позже date_to')
    result = build_report(term.strip(), date_from, date_to, limit)
    result['requested_by'] = user['sub']
    if not trace:
        result.pop('trace')
    return result
```

Приложение В. API Gateway: app\gateway\main.py

```
"""API Gateway: единая точка входа. Выдаёт JWT, проверяет его и маршрутизирует вызовы к сервисам
    лабораторных."""
import logging
import os
from datetime import date


import httpx
from fastapi import Depends, FastAPI, HTTPException, Query, Request, status
from fastapi.responses import JSONResponse
from fastapi.security import OAuth2PasswordRequestForm


from app.common.web import utf8_json
from app.common.security import current_user, issue_token, load_users, verify_password


logging.basicConfig(level=logging.INFO, format='%(asctime)s %(name)s %(levelname)s %(message)s')
logging.getLogger('httpx').setLevel(logging.WARNING)      # без строки на каждый внутренний HTTP-запрос
log = logging.getLogger('gateway')
LAB1_URL = os.environ.get('LAB1_URL', 'http://lab1:8001')
USERS = load_users()


app = FastAPI(title='API Gateway — Проектирование архитектуры ПО', version='1.0.0',
              description='Аутентификация по JWT (кнопка Authorize) и вызов аналитических запросов
                  лабораторных работ.')
app.middleware('http')(utf8_json)




@app.get('/health', tags=['service'])
def health():
    try:
        lab1 = httpx.get(f'{LAB1_URL}/health', timeout=3).json()['status']
    except httpx.HTTPError:
        lab1 = 'unavailable'
    return {'status': 'ok', 'lab1': lab1}




@app.post('/auth/login', tags=['auth'], summary='Получить JWT по логину и паролю')
def login(form: OAuth2PasswordRequestForm = Depends()):
    user = USERS.get(form.username)
    if not user or not verify_password(form.password, user['hash']):
        log.warning('неудачный вход: %s', form.username)
        raise HTTPException(status.HTTP_401_UNAUTHORIZED, 'Неверный логин или пароль',
                            headers={'WWW-Authenticate': 'Bearer'})
    log.info('выдан токен: %s (%s)', form.username, user['role'])
    return issue_token(form.username, user['role'])




@app.get('/auth/me', tags=['auth'], summary='Сведения из токена')
def me(user: dict = Depends(current_user)):
    return user




@app.get('/api/lab1/report', tags=['lab1'],
         summary='ЛР №1: 10 студентов с минимальной посещаемостью лекций, содержащих термин, за период')
def lab1_report(request: Request,
                term: str = Query(..., min_length=2, max_length=200, examples=['партиционирование']),
                date_from: date = Query(..., examples=['2026-02-09']),
                date_to: date = Query(..., examples=['2026-04-05']),
                limit: int = Query(10, ge=1, le=100), trace: bool = Query(False),
                user: dict = Depends(current_user)):
    # токен уже проверен шлюзом; передаём его сервису, который проверяет подпись ещё раз
    params = {'term': term, 'date_from': date_from.isoformat(), 'date_to': date_to.isoformat(),
              'limit': limit, 'trace': str(trace).lower()}
    try:
        resp = httpx.get(f'{LAB1_URL}/report', params=params, timeout=30,
                         headers={'Authorization': request.headers['authorization']})
    except httpx.HTTPError as e:
```

```
        raise HTTPException(status.HTTP_502_BAD_GATEWAY, f'Сервис ЛР №1 недоступен: {e}')
    log.info('lab1 report: user=%s term=%r %s..%s -> %s', user['sub'], term, date_from, date_to,
        resp.status_code)
    try:
        return JSONResponse(resp.json(), status_code=resp.status_code)
    except ValueError:   # сервис ответил не JSON (внутренняя ошибка)
        raise HTTPException(status.HTTP_502_BAD_GATEWAY, f'Сервис ЛР №1 вернул {resp.status_code}:
            {resp.text[:200]}')
```

Приложение Г. JWT и пароли: app\common\security.py, app\common\web.py

```
"""JWT-аутентификация, общая для шлюза и сервиса ЛР №1.


Шлюз выдаёт токен (POST /auth/login) и проверяет его на каждом вызове API; сервис ЛР №1 проверяет
тот же токен повторно (общий секрет HS256), поэтому вызвать его в обход шлюза без токена нельзя.
Пароли пользователей хранятся только в виде PBKDF2-SHA256 с солью (переменная GATEWAY_USERS).
"""
from __future__ import annotations


import base64
import hashlib
import hmac
import os
import secrets
import time


import jwt
from fastapi import Depends, HTTPException, status
from fastapi.security import OAuth2PasswordBearer


SECRET = os.environ.get('JWT_SECRET', '')
ALGORITHM = 'HS256'
ISSUER = 'papo-gateway'
TTL_MIN = int(os.environ.get('JWT_TTL_MIN', '60'))
oauth2_scheme = OAuth2PasswordBearer(tokenUrl='/auth/login')




def hash_password(password: str, iterations: int = 200_000) -> str:
    """Строка вида pbkdf2_sha256:<итерации>:<соль>:<хеш> (без символа $, чтобы не мешать .env/compose)."""
    salt = secrets.token_bytes(16)
    dk = hashlib.pbkdf2_hmac('sha256', password.encode(), salt, iterations)
    return f'pbkdf2_sha256:{iterations}:{base64.b64encode(salt).decode()}:{base64.b64encode(dk).decode()}'




def verify_password(password: str, stored: str) -> bool:
    try:
        algo, iters, salt, digest = stored.split(':')
        if algo != 'pbkdf2_sha256':
            return False
        dk = hashlib.pbkdf2_hmac('sha256', password.encode(), base64.b64decode(salt), int(iters))
        return hmac.compare_digest(dk, base64.b64decode(digest))   # сравнение за постоянное время
    except ValueError:
        return False




def load_users() -> dict[str, dict]:
    """GATEWAY_USERS = 'логин|роль|хеш;логин|роль|хеш'"""
    users = {}
    for item in filter(None, os.environ.get('GATEWAY_USERS', '').split(';')):
        login, role, pw_hash = item.split('|')
        users[login.strip()] = {'role': role.strip(), 'hash': pw_hash.strip()}
    return users




def issue_token(username: str, role: str) -> dict:
    now = int(time.time())
    claims = {'sub': username, 'role': role, 'iss': ISSUER, 'iat': now, 'exp': now + TTL_MIN * 60}
    return {'access_token': jwt.encode(claims, SECRET, algorithm=ALGORITHM), 'token_type': 'bearer',
            'expires_in': TTL_MIN * 60}




def decode_token(token: str) -> dict:
```

```
    return jwt.decode(token, SECRET, algorithms=[ALGORITHM], issuer=ISSUER, options={'require': ['exp',
        'sub']})



def current_user(token: str = Depends(oauth2_scheme)) -> dict:
    try:
        return decode_token(token)
    except jwt.ExpiredSignatureError:
        detail = 'Срок действия токена истёк'
    except jwt.PyJWTError as e:
        detail = f'Недействительный токен: {e}'
    raise HTTPException(status.HTTP_401_UNAUTHORIZED, detail=detail, headers={'WWW-Authenticate': 'Bearer'})
"""Общие настройки HTTP для шлюза и сервиса."""



async def utf8_json(request, call_next):
    """Явная кодировка в Content-Type ответов JSON: без неё часть клиентов (Invoke-RestMethod в
    Windows PowerShell 5.1) читает тело как ISO-8859-1 и портит кириллицу."""
    resp = await call_next(request)
    if resp.headers.get('content-type') == 'application/json':
        resp.headers['content-type'] = 'application/json; charset=utf-8'
    return resp
```

Приложение Д. Dockerfile, docker-compose.yml, requirements.txt

```
# Один образ для шлюза и сервиса ЛР №1: разные контейнеры запускают разные приложения (command в compose)
FROM python:3.12-slim
ENV PYTHONDONTWRITEBYTECODE=1 PYTHONUNBUFFERED=1
WORKDIR /srv
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY app ./app
USER nobody
# ЛР №1: API Gateway + сервис аналитического запроса. Хранилища — стенд ПР №1 (сеть practice01_default),
# данные — структура ПР №3 (university_system). Запуск: docker compose up -d --build
name: lab1


x-logging: &default-logging
  driver: json-file
  options: { max-size: "10m", max-file: "3" }


services:
  lab1:
    build: .
    image: papo-lab:1.0
    container_name: lab1-service
    command: uvicorn app.lab1.main:app --host 0.0.0.0 --port 8001
    env_file: .env
    expose: ["8001"]          # наружу не публикуется: доступ только через шлюз
    healthcheck:
      test: ["CMD", "python", "-c", "import urllib.request;
          urllib.request.urlopen('http://localhost:8001/health')"]
      interval: 15s
      timeout: 5s
      retries: 5
    mem_limit: 256m
    restart: unless-stopped
    logging: *default-logging


  gateway:
    build: .
    image: papo-lab:1.0
    container_name: api-gateway
    command: uvicorn app.gateway.main:app --host 0.0.0.0 --port 8000
    env_file: .env
    ports: ["8000:8000"]      # единственная точка входа: http://localhost:8000/docs
    depends_on:
      lab1:
        condition: service_healthy
    healthcheck:
      test: ["CMD", "python", "-c", "import urllib.request;
          urllib.request.urlopen('http://localhost:8000/health')"]
      interval: 15s
      timeout: 5s
      retries: 5
    mem_limit: 256m
    restart: unless-stopped
    logging: *default-logging


networks:
  default:
```

```
    name: practice01_default
    external: true
fastapi==0.115.6
uvicorn==0.34.0
httpx==0.28.1
PyJWT==2.10.1
python-multipart==0.0.20
psycopg[binary]==3.2.3
redis==5.2.1
```

Приложение Е. Независимая сверка: tests\verify.py

```
"""Независимая проверка расчёта ЛР №1.


Берёт из ответа шлюза (trace=true) идентификаторы лекций, найденных Elasticsearch, и пересчитывает
посещаемость другим SQL-запросом напрямую в PostgreSQL — без условия на партиции и без кэша. Топ-10
сервиса и пересчёт должны совпасть построчно.  Запуск: python -P tests/verify.py [термин] [с] [по]
"""
import json, subprocess, sys, urllib.parse, urllib.request
B = 'http://localhost:8000'
term, d1, d2 = (sys.argv[1:4] + ['API Gateway', '2026-02-09', '2026-05-31'][len(sys.argv[1:4]):])


tok = json.loads(urllib.request.urlopen(urllib.request.Request(
    B + '/auth/login', data=urllib.parse.urlencode({'username': 'teacher', 'password':
        'teacher-pass'}).encode())).read())['access_token']
q = urllib.parse.urlencode({'term': term, 'date_from': d1, 'date_to': d2, 'trace': 'true'})
rep = json.loads(urllib.request.urlopen(urllib.request.Request(B + '/api/lab1/report?' + q,
                                                               headers={'Authorization': 'Bearer ' +
                                                                   tok})).read())
ids = rep['trace'][0]['details']['lecture_ids']
service = [(x['student']['student_card_number'], x['attended'], x['total'], x['attendance_pct']) for x in
    rep['students']]


sql = f"""
SELECT st.student_card_number || '|' || sum(CASE WHEN a.is_present THEN 1 ELSE 0 END) || '|' || count(*) ||
    '|' ||
       round(100.0 * sum(CASE WHEN a.is_present THEN 1 ELSE 0 END) / count(*), 1)
FROM students st
JOIN attendance a ON a.student_id = st.id
JOIN schedule s   ON s.id = a.schedule_id
WHERE s.lecture_id IN ({', '.join("'" + i + "'" for i in ids)})
  AND s.scheduled_date BETWEEN '{d1}' AND '{d2}'
GROUP BY st.id, st.student_card_number
ORDER BY sum(CASE WHEN a.is_present THEN 1 ELSE 0 END)::numeric / count(*), count(*) DESC, st.id
LIMIT 10;"""
out = subprocess.run(['docker', 'exec', '-i', 'practice01-postgres', 'psql', '-U', 'admin', '-d',
    'university_system',
                      '-tA'], input=sql, capture_output=True, text=True, encoding='utf-8').stdout.split()
direct = [(c, int(a), int(t), float(p)) for c, a, t, p in (r.split('|') for r in out)]
print(f'термин: {term}; период {d1}…{d2}; лекций из Elasticsearch: {len(ids)}')
print(f'{"№":>2}  {"сервис (зачётка посещ/всего %)":<34} {"прямой SQL":<34} совпадает')
ok = True
for i, (a, b) in enumerate(zip(service, direct), start=1):
    same = a == b
    ok &= same
    print(f'{i:>2}  {a[0]} {a[1]:>2}/{a[2]:<3} {a[3]:>5}%{"":13} {b[0]} {b[1]:>2}/{b[2]:<3} {b[3]:>5}%{"":13}
        {"да" if same else "НЕТ"}')
print('ИТОГ:', 'расчёт сервиса подтверждён' if ok and len(service) == len(direct) else 'РАСХОЖДЕНИЕ')
sys.exit(0 if ok else 1)
```